https://hjfy.top/arxiv/2212.08013

FlexiViT：适用于所有 Patch 大小的单一模型

Lucas Beyer[⋆1] Pavel Izmailov[⋆1,3] Alexander Kolesnikov[⋆1] Mathilde Caron[⋆2] Simon Kornblith[⋆1]
Xiaohua Zhai[⋆1] Matthias Minderer[⋆1] Michael Tschannen[⋆1] Ibrahim Alabdulmohsin[⋆1] Filip Pavetic[⋆1]

Google Research

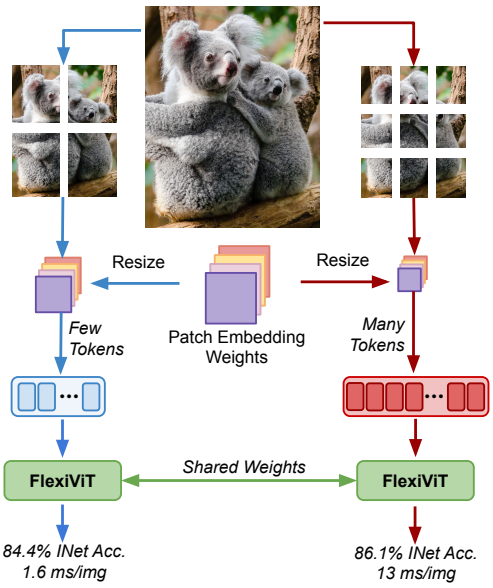

图 1. FlexiViT 是一种标准的 ViT 模型，在训练过程中会看到随机化的 patch 大小，因此序列长度也会变化。每个 patch 大小的嵌入权重会自适应调整，而模型权重在所有 patch 大小之间保持不变。

Abstract

视觉 Transformers 通过将图像分割成小块来将其转换为序列。这些小块的大小控制着速度与准确率之间的权衡，较小的块在更高的计算成本下带来更高的准确率，但改变块大小通常需要重新训练模型。在本文中，我们展示了在训练时简单地随机化块大小，可以生成一组在广泛块大小范围内表现良好的权重，从而使得在部署时能够根据不同的计算预算调整模型。我们对这一模型进行了广泛评估，称之为 FlexiViT，涵盖分类、图文检索、开放世界检测、全景分割和语义分割等多种任务，结论是它在相同设置下通常与标准 ViT 模型相当，有时甚至优于后者。因此，FlexiViT 训练是对 ViT 的一种简单即插即用的改进，使得大多数依赖 ViT 骨干架构的模型能够轻松增加计算自适应能力。代码和预训练模型可在github.com/google-research/big_vision获取。

1. 引言

视觉 Transformer（ViTs）将图像切割成不重叠的补丁，并在这些补丁生成的 token 上执行所有计算。这种“补丁化”过程标志着与之前主导的卷积神经网络（CNN）方法 [33] 的重大转变，后者使用小的局部且通常重叠的滤波器处理图像。补丁化开启了新的能力，例如（随机）丢弃图像补丁 token [10,21,47,56,65]，为新任务添加专用 token [57,59]，或将图像 token 与其他模态的 token 混合 [1,41,68]。

尽管分块化对于 ViT 模型的重要性不言而喻，但块大小所扮演的角色却鲜少受到关注。在原始 ViT 论文 [16] 中，探讨了三种块大小（32× 32、16× 16 和 14× 14 像素），而许多后续研究则将块大小固定为 16× 16 像素 [57,58,69]。在本研究中，我们证明了块大小提供了一个简单而有效的杠杆，可以在不改变模型参数化的情况下，调整模型的计算量和预测性能。例如，ViT-

⋆ 所有作者均做出了重要的技术贡献。
Lucas 发起并领导了该项目。
[1] Google Research, Brain Team。
[2] Google Research。
[3] 在 Google Brain 工作期间完成，同时为纽约大学博士研究生。

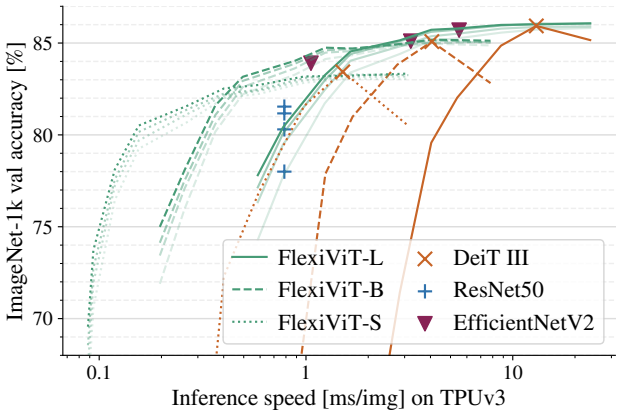

图 2. FlexiViT **在** ImageNet-1k **上的结果**。我们基于 DeiT III 在 ImageNet-1k 上训练了三个 FlexiViT，并展示了在不同 patch 大小下评估时的速度-准确率权衡。每条曲线对应一个具有单一权重集的模型在不同 patch 大小下的运行结果。我们还评估了 DeiT III 在不同 patch 大小下的表现，以展示 ViT 的天然不灵活性。EfficientNet-v2 的数据来自 [55]，ResNet50 的数据来自 [5]，后者是从 ImageNet-21k 预训练教师模型蒸馏得到的。FlexiViT 训练了 1000 轮次，但运行 600、300 和 90 轮次的结果以阴影曲线显示，表明长时间训练主要对短序列情景有益，并非绝对必要。

B/8 模型在 ImageNet1k 上以 156 GFLOPs 和 85 M 参数实现了 85.6% 的 top-1 准确率，而 ViT-B/32 模型仅以 8.6 GFLOPs 和 87 M 参数达到了 79.1% 的准确率。尽管在性能和计算量上存在显著差异，这些模型本质上拥有相同的参数化。然而，标准的 ViT 模型仅在它们被训练时的块大小下表现良好。因此，调整块大小需要对模型进行完整的重新训练。

为了克服这一限制，我们提出了 FlexiViT，一种灵活的 ViT，它在广泛的 patch 大小范围内匹配或优于标准的固定 patch ViT，且无需额外成本。为了训练 FlexiViT，我们在训练过程中随机化 patch 大小，并自适应地调整位置嵌入和 patch 嵌入参数以适应每个 patch 大小，如图 1所示。这些简单的修改已经足以实现强大的性能，但我们也提出了一种优化的调整操作和基于知识蒸馏的训练过程，以取得更好的结果。

我们展示了 FlexiViT 模型在诸多下游任务中的高效性，例如图像分类、迁移学习、全景与语义分割、图文检索以及开放世界识别，并提供了将现有基于 ViT 的训练设置*灵活化*的通用方案。此外，我们证明了骨干网络的*灵活性*——即在不同 patch 尺寸下均表现出色——即便在固定 patch 尺寸下进行微调后，这一特性也往往得以保留。我们利用这一观察结果，实现了资源高效的迁移学习：以较大 patch 尺寸低成本地微调模型，随后以较小 patch 尺寸部署，以获得强劲的下游任务表现。我们还进一步展示了灵活 patch 尺寸可用于加速预训练过程。

为阐明 FlexiViT 的效能，我们深入分析了该模型的表示。研究发现，不同 patch 大小下的表示往往相似，尤其是在更深层中。最终，我们证明 FlexiViT 在控制 ViT 模型性能与计算量权衡方面，超越了其他架构设计方式。

2. 相关工作

最近的一些工作通过利用分块技术来提高 ViT 的效率。一些方法建议在训练过程中以随机 [21] 或结构化 [10] 的方式移除 token。另一些方法旨在量化 token 的重要性，并在 [47, 65] 或 [56] 训练期间移除最重要性最低的 token。[60] 在训练时使用不断增加的 token 数量的级联 Transformer，以便在推理时提前退出。相反，我们始终保留所有 token，并不丢弃任何信息。在未来的工作中，可能可以将这些方法与 FlexiViT 相结合。

另一相关研究方向探讨了在训练过程中改变输入分辨率，通常是为了加速预训练 [38, 55, 58, 62] 或作为 ViTs 自监督学习的数据增强 [35]。我们未探索 Flexi 训练在数据增强方面的影响，以避免研究范围过于膨胀。上述模型均仅在其单一、最终分辨率下工作，而我们的目标是拥有一个在所有训练过的 patch 大小下都能表现良好的单一模型。

与我们的方法更为相似的是，神经架构搜索（NAS）领域正趋向于训练一个“超级网络”，从中可以提取出形状各异的“子网络” [8, 19, 67]。由于这些工作旨在改变模型的大部分或全部维度，它们通常涉及多种专门的架构添加。SuperViT [36] 与 FlexiViT 最为相关，因为它以多种尺度对图像进行分块处理，将所有块传递给 ViT，同时随机丢弃 token [21] 以减少序列长度。与上述工作相比，我们仅专注于 ViT 的块大小，这使得我们能够受益于现有的预训练模型、未来的 ViT 改进，并且可以轻松地融入任何现有的训练流程。

Matryoshka 表示学习 [32] 提出了一种训练模型的方法，其输出向量包含有意义的子向量。这可以被视为 FlexiViT 的补充。

3. 使 ViT 更具灵活性

在本节中，我们表明标准的 ViT 模型不够灵活，并在监督图像分类设置中引入了 FlexiViT 模型和训练过程。我们在本节的所有实验均在公开的 ImageNet-21k 数据集 [49] 上进行。我们使用基础（ViT-B）规模模型和 无正则化 light2 情景 from [53]，并按照 [39] 训练模型 90 轮次。

3.1. 背景与符号

FlexiViT 基于 Vision Transformer（ViT）架构 [16]。在此，我们简要描述 ViT 架构并引入必要的符号。

考虑一张图像 $x \in \mathbb{R}^{h \times w \times c}$，其中 (h, w, c) 分别表示宽度、高度和通道数。ViT 首先将输入图像分割成一系列 s 个图像块 $x_i \in \mathbb{R}^{p \times p \times c}$，其中 $i \in \{1, \ldots, s\}$。我们将这一过程称为 图像块化，并在图 1 中进行了展示。序列长度 $s = \lfloor h/p \rfloor \cdot \lfloor w/p \rfloor$ 是图像块化后的图像块（或 token）数量，它控制了 ViT 的计算量：自注意力的计算复杂度为 $\mathcal{O}(s^2) = \mathcal{O}(h^4) = \mathcal{O}(w^4)$，即与图像高度（或宽度）的四次方成正比。

接下来，我们为每个 x_i 计算 补丁嵌入 $e_i = (e_i^k)_{k=1}^d \in \mathbb{R}^d$：$e_i^k = \langle x_i, \omega_k \rangle = \mathrm{vec}(x_i)^T \mathrm{vec}(\omega_k)$，其中 $\omega_k \in \mathbb{R}^{p \times p \times c}$ 是补丁嵌入的权重，$\langle \cdot, \cdot \rangle$ 表示点积，vec 是将多维数组展平为向量的操作。最后，我们将学成的位置嵌入 $\pi_i \in \mathbb{R}^d$ 添加到补丁嵌入 $t_i = e_i + \pi_i$ 中。然后，我们将 s token 序列 t_i 作为输入传递给 Transformer 编码器，如图 1 所示。

总之，对于给定的图像大小 $h \times w$，patch 大小 p 决定了 Transformer 模型的输入序列长度 s：较小的 patch 大小对应于较长的输入序列和较慢但更具表达力的模型。根据 [16]，我们将 ViT 模型表示为 ViT-$\mathcal{S}$ /p，其中 $\mathcal{S} \in \{\text{S, M, B, L, ...}\}$ 表示模型规模（小、中、基础、大等），而 p 表示 patch 大小。需要注意的是，模型中仅有两个部分的参数向量依赖于 patch 大小：patch 嵌入权重 ω_k 和位置嵌入 π。在接下来的章节中，我们将开发一种 灵活的 ViT 模型，该模型可同时适用于任何 patch 大小。

3.2. 标准视觉 Transformer（ViTs）缺乏灵活性

我们首先展示了对标准预训练 ViT 模型在不同 patch 大小下进行评估时性能较差。为了改变 patch 大小，我们简单地使用双线性插值调整 patch 嵌入权重 ω 和位置嵌入 π。[2]. 对于位置嵌入，这种调整大小的策略已在原始 ViT 论文 [16] 中提出，以便在更高分辨率下进行微调。

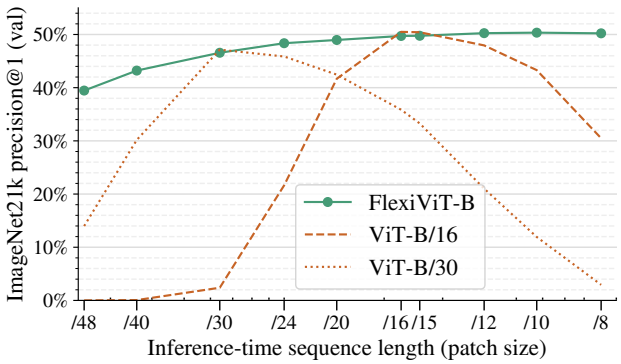

图 3. **标准 ViTs 在 patch 大小上缺乏灵活性**。然而，FlexiViT 能够在无需性能损失的情况下训练它们实现灵活性。

Algorithm 1 Minimal FlexiViT pseudo-implementation.

```
model = ViT(...)
for batch in data:
  ps = np.random.choice([8, 10, ..., 40, 48])
  logits = model(batch["images"], (ps, ps))
  # [...] backprop and optimize as usual

class ViT(nn.Module):
  def __call__(self, image, patchhw):
    # Patchify, flexibly:
    w = self.param("w_emb", (32, 32, 3, d))
    b = self.param("b_emb", d)
    w = resize(w, (*patchhw, 3, d))
    x = conv(image, w, strides=patchhw) + b
    # Add flexible position embeddings:
    pe = self.param("posemb", (7, 7, d))
    pe = resize(pe, (*x.shape[1:3], d))
    return TransformerEncoder(...)(x + pe)
```

Notes: Changes to existing code highlighted via violet background.

结果如图 3 所示，我们可以看到标准 ViT 模型的性能（虚线和点线）随着推理时 patch 大小与训练时使用的 patch 大小偏离而迅速下降。

3.3. 训练灵活的视觉 Transformer 模型

在图 3中，我们还展示了我们的 FlexiViT-B 模型（实线）的性能，该模型在评估其训练 patch 大小时与 ViT-B/16 和 ViT-B/30 相匹配，并且对于所有其

[2] tf.image.resize(input, res, method='bilinear')

他 patch 大小都显著优于它们。该模型在与 ViT-B/16 和 ViT-B/30 模型相同的情景下进行训练，不同之处在于在训练的每一步中，patch 大小是从一组预定义的 patch 大小中随机均匀选择的。[3] 为此，需对模型和训练代码进行两处小调整。

首先，模型需要为 ω 和 π 定义一个*潜在参数形状*。可学习参数具有该形状，并在模型的前向传播过程中动态调整大小。我们在附录 B 中展示了这些潜在可学习参数的确切形状并不重要，并且在所有实验中，我们使用 32×32 作为补丁的潜在大小，7×7 作为位置嵌入的潜在大小。

其次，为了拥有多种能够完美平铺图像的补丁大小，我们使用了 240^2 px 的图像分辨率，这允许补丁大小为 $p \in \{240, 120, 60, 48, 40, 30, 24, 20, 16, 15, 12, 10, 8, 6, 5, 4, 2, 1\}$，其中我们使用了 48 到 8 之间的所有大小，包括 48 和 8。[4] 在每次迭代中，我们从这些块大小上的均匀分布 $\mathcal{P}$ 中采样 p。

这些是使现有 ViT *灵活化*所需的所有更改。Algorithm 1 对其进行了总结。

请注意，改变补丁大小与改变图像大小相关，但并不完全相同。补丁大小的改变纯粹是对模型的调整，而改变图像大小可能会显著减少可用信息。这一区别在章节 3.4 中进一步探讨。

我们在第 7 节中探讨了两种灵活化 ViTs 的替代方法：灵活深度和灵活补丁步幅。两者各有优点，但补丁大小效果最佳。

3.4. 如何调整补丁嵌入的 Size

考虑输入图像的一个补丁 $x \in \mathbb{R}^{p \times p}$ 和补丁嵌入权重 $\omega \in \mathbb{R}^{p \times p}$，并假设我们处理的是非负值的简单场景。如果我们使用双线性插值调整补丁和嵌入权重的大小，生成的 token 的范数将会有显著差异；例如 $\langle x, \omega \rangle \approx \frac{1}{4}\langle \text{resize}_p^{2p}(x), \text{resize}_p^{2p}(\omega) \rangle$。我们假设这种 token 范数的剧烈变化是 ViT 缺乏灵活性的部分原因，也是一种阻碍单一 FlexiViT 学习的归纳偏置。理想情况下，只要在调整大小过程中没有信息丢失，调整输入 x 和嵌入 ω 大小后的补丁嵌入 $e_i = \langle x, \omega \rangle$ 应该保持不变。

[3]在大多数实验中，我们均匀地采样补丁大小。一些早期实验使用了略微偏好中等大小补丁的分布。后续实验表明，这种分布影响甚微（附录 C）。因此，我们未重新进行早期实验。

[4]完美的平铺可能并非绝对必要，使用任意补丁大小并忽略图像的一小部分边缘或许也是可行的。为简化起见，我们专注于完美平铺的情景。

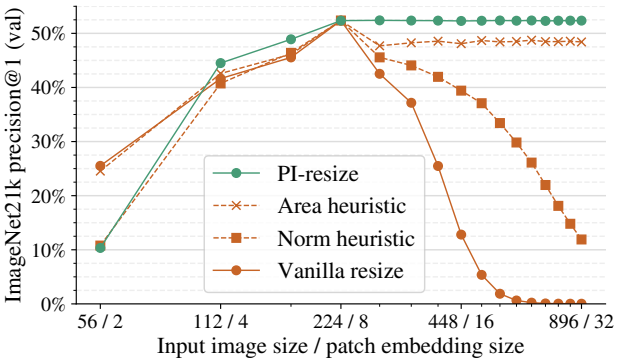

图 4. **多种“调整大小”ViT 的方法**。我们从 [53] 加载一个在 224^2 px 上训练的 ViT-B/8，通过相同的因子调整嵌入和输入图像的大小，并计算验证准确率。PI-resize 是唯一在放大时仍保持准确的方法。

实现这一等式的一种方法是在 token 嵌入后立即对其进行归一化，无论是显式地还是通过使用 LayerNorm [2] 模块。然而，这种方法需要改变模型架构，并且与现有的预训练 ViTs 不兼容。此外，它并不能确切地保留 patch 嵌入。正如我们将展示的，有一种更原则性的方法来实现这一目标，它与现有的预训练模型兼容，且无需任何架构上的改动。

首先，我们注意到在第 3.2 节中介绍的线性调整大小操作可以用一个线性变换来表示：

$$\text{resize}_p^{p_*}(o) = B_p^{p_*} \, \text{vec}(o), \tag{1}$$

其中 $o \in \mathbb{R}^{p \times p}$ 是任意输入，而 $B_p^{p_*} \in \mathbb{R}^{{p_*}^2 \times p^2}$。我们独立地调整多通道输入 o 的通道大小。

直观上，我们希望找到一组新的 patch-embedding 权重 $\hat{\omega}$，使得调整大小后的 patch 的 token 与原始 patch 的 token 相匹配。形式上，我们希望解决以下优化问题：

$$\hat{\omega} \in \arg\min_{\hat{\omega}} \mathbb{E}_{x \sim \mathcal{X}} \left[(\langle x, \omega \rangle - \langle Bx, \hat{\omega} \rangle)^2 \right], \tag{2}$$

其中 $B = B_p^{p_*}$ 和 $\mathcal{X}$ 是补丁上的某种分布。在我们增加补丁大小的情况下，即 $p_* \geq p$，我们可以使用 $\hat{\omega} = P\omega$，其中 $P = B(B^T B)^{-1} = (B^T)^+$ 是 B^T 的伪逆：

$$\langle Bx, \hat{\omega} \rangle = x^T B^T B (B^T B)^{-1} w = x^T w = \langle x, w \rangle. \tag{3}$$

这样我们为所有 x 精确匹配了补丁嵌入。

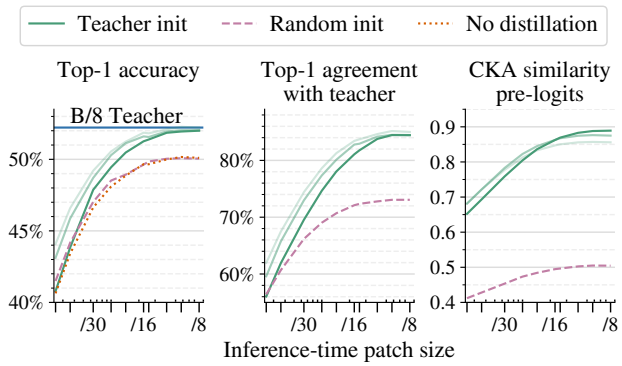

图 5. **初始化影响**在蒸馏到 FlexiViT 时的效果。

在下采样的情况下，即当 $p_* < p$ 时，式 (2) 中优化问题的解通常依赖于块分布 $\mathcal{X}$。在附录 A.2 中，我们展示了对于 $\mathcal{X} = \mathcal{N}(0, I)$，我们恢复了伪逆 $\hat{\omega} = P\omega = (B^T)^+\omega$ 作为最优解。[5]. 综上所述，我们将 PI-resize（伪逆调整大小）定义为：

$$\text{PI-resize}_p^{p_*}(w) = \left((B_p^{p_*})^T\right)^+ \text{vec}(\omega) = P_p^{p_*}\,\text{vec}(\omega), \quad (4)$$

其中 $P_p^{p_*} \in \mathbb{R}^{{p_*}^2 \times p^2}$ 是对应于 PI-resize 变换的矩阵。PI-resize 操作调整了 patch embedding 权重的大小，作为双线性 resize 操作的逆。

为了实验验证 PI-resize 的有效性，并将其与包括标准线性调整大小在内的几种替代启发式方法进行比较，我们从 [53] 加载了一个预训练的 ViT-B/8 模型，并在调整图像和模型大小后对其进行评估，从而保持其序列长度 $s = (224/8)^2 = 784$。结果显示在图 4中，表明 PI-resize 在上采样时几乎保持性能不变，在下采样时性能下降得较为平缓。在所有情况下，没有任何启发式方法能像精心设计的 PI-resize 那样全面有效。

为求全面，在附录 A.1 中，我们实验比较了在不考虑保持模型兼容性的情况下处理可变补丁大小的其他方法。这些方法包括固定规范化、LayerNorm 以及为每种补丁大小学习分离的参数 ω。添加 LayerNorm 效果最佳，但除此之外，PI-resize 和双线性插值也是最佳技术之一。

3.5. 与知识蒸馏的联系

知识蒸馏 [24] 是一种流行的技术，其中通常较小的学生模型被训练以模仿通常较大的教师模型的预测。与标准的标签监督训练 [5, 12, 64] 相比，这可以显著提高学生模型的性能。

最近研究表明，知识蒸馏对应着一个比标准监督训练更为复杂的优化问题 [5, 52]，而将学生模型初始化为接近教师模型的状态可以简化并缓解这一难题 [52]。遗憾的是，由于教师模型通常拥有与学生模型不同（且更大）的架构，这一解决方案并不实用 [5]。然而，借助 FlexiViT，我们能够用强大的 ViT 教师模型的权重初始化学生 FlexiViT 模型，从而显著提升蒸馏性能。

除非另有说明，本文中剩余实验所使用的模型是一个从强大的 ViT-B/8 模型 [53] 初始化并蒸馏得到的 FlexiViT-B。初始化时，我们将教师模型的块嵌入权重 PI 调整为 32×32 ，并双线性重采样其位置嵌入至 7×7 。然后我们按照 FunMatch [5] 的方法训练学生模型，通过随机块大小最小化教师模型和学生 FlexiViT 预测之间的 KL 散度：

$$\mathbb{E}_{x \in \mathcal{D}} \mathbb{E}_{p \sim \mathcal{P}}\, \text{KL}\left(f_{\text{FlexiViT}}(x, p) || f_{\text{ViT-B/8}}(x)\right), \quad (5)$$

其中 $f_{\text{FlexiViT}}(x, p)$ 是 FlexiViT 模型在输入 x 上关于类别的分布，其补丁大小为 p，$f_{\text{ViT-B/8}}(x)$ 是教师模型在完全相同输入上的预测分布，$\mathcal{D}$ 是包含随机翻转、裁剪和混合的训练数据分布，而 $\mathcal{P}$ 是用于训练 FlexiViT 模型的补丁大小分布。

图 5 比较了使用教师初始化进行蒸馏与随机初始化以及从标签进行监督训练的效果。该比较进行了 90 轮次，展示了 FlexiViT 这种独特初始化能力的显著优势。由于蒸馏需要耐心 [5, 57]，我们额外运行了 300 和 1000 轮次，如图中浅绿色曲线所示。FlexiViT 在小尺寸补丁上匹配了教师的性能，而在最大尺寸补丁上，教师初始化显著提高了准确率。在接下来的部分中，我们使用训练了 300 轮次的 FlexiViT，并在相同情景（包括初始化）下训练了两个固定的 ViT-B/30 和 ViT-B/16 模型作为基准。

[5]我们也可以针对数据中的补丁分布而非 $\mathcal{X}$，生成一种依赖于数据的调整大小操作。在初步实验中，我们并未观察到这种方法带来的显著优势。

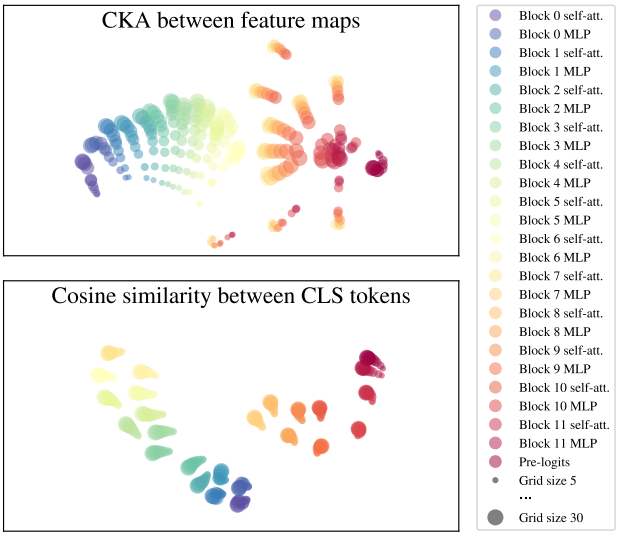

图 6. 不同网格大小下网络层中间 t-SNE **可视化**。颜色表示不同层；点大小反映不同网格大小。

3.6. FlexiViT 的内部表示

FlexiViT 是否以相似的方式处理具有不同 patch 大小的输入？我们通过分析模型的内部表示来研究这个问题。我们应用了小批量中心核对齐（CKA）[14,29,42]，这是一种广泛用于比较神经网络内部和跨网络表示的方法。为了可视化目的，我们对 CKA/余弦相似度应用了反余弦变换，将其转换为适当的度量 [61]，然后执行 t-SNE。

结果如图 6所示。从第一层到第 6 块的 MLP 子层，特征图表示在不同网格大小下相似。在第 6 块的 MLP 子层，层表示开始发散，但在最后一个块再次收敛。相比之下，CLS token 表示在不同网格大小下保持一致。因此，尽管 FlexiViT 大部分内部表示因网格大小而异，但输出表示通常保持一致。

4. 使用预训练的 FlexiViTs 模型

我们已经展示了 ViTs 可以灵活训练而不会显著损失上游性能。接下来，我们验证了预训练的 FlexiViTs 在迁移到其他任务时，仍然与单个固定 patch 大小的 ViTs 相当。我们通过在迁移过程中将预训练的 FlexiViT 的 patch 大小固定为 16^2 或 30^2 来进行检查。我们将 FlexiViT 与 ViT-B/16 和 ViT-B/30 模型进行比较，这些模型使用与 FlexiViT 相同的蒸馏设置进行预训练（第 3.5节），但具有固定的 patch 大小。我们在以下多样化任务集上执行此迁移。

对于每项任务，我们在附录 E 中提供了更多细节以及大量结果，所有内容都传达了相同的关键信息。

Classification　我们在小规模（Pet [44], Flowers [43]）和中等规模（CIFAR10, CIFAR100 [31], Food101 [7], SUN397 [63]）分类数据集上进行微调，遵循 [16] 的设置，在 240^2 px 分辨率下进行。

Locked-image Tuning (LiT)　We follow [70] to train a text model contrastively [25,46] for the frozen FlexiViT, which we evaluate in terms of 0-shot classification and retrieval.

Open-vocabulary detection　我们使用 OWL-ViT [40] 测试 FlexiViT 在目标检测中的迁移能力，这是一种基于图像-文本模型（如 LiT 或 CLIP）的开放词表目标检测器 [46]。我们在 LVIS [20] 上评估其 zero-shot 开放词表检测性能。

Panoptic segmentation　通用视觉模型（UViM）是一种面向视觉的通用建模方法 [28]。我们在 COCO 全景分割数据集 [26,37] 上训练 UViM，并使用 FlexiViT 作为 UViM 中图像编码器的初始化。

Semantic segmentation　我们按照 Segmenter 的线性解码器设置转移到语义分割 [54]。我们报告单尺度评估的平均 IoU，并在 Cityscapes [13] 和 ADE-20k [71] 上进行评估。

4.1. 结果

这些迁移实验的结果总结于图 7。在多样化的任务集中，单个 FlexiViT 模型大致与两个固定 ViT 模型相当，在大尺寸补丁时仅略微落后，而在较小尺寸补丁时则带来小幅或显著提升。

这些结果证实，使用预训练的 FlexiViT 相较于为不同 patch 大小预训练多个 ViT 并无显著劣势。

4.2. 资源高效传输通过灵活性实现

FlexiViT 提供了一种新方法，使迁移学习更加资源高效，节省加速器内存和计算资源。这之所以可能，是因为令人惊讶的是，*即使在固定补丁大小下进行迁移后，灵活性仍然在很大程度上得以保留*。因此，我们可以使用较大的输入补丁（较小的输入网格）进行廉价的迁移训练，但随后可以使用较小的补丁大小（较大的输入网格）部署生成的模型。

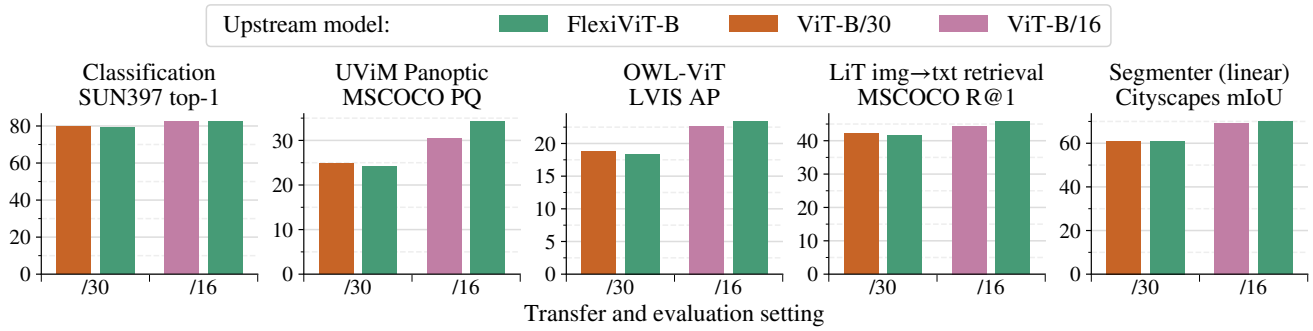

图 7. **使用预训练的** FlexiViT。我们在多种下游计算机视觉任务中，以两种补丁大小使用灵活预训练的 FlexiViT-B 模型，并验证其性能与在该补丁大小下预训练的普通（不灵活）ViT 模型相同或更优。这些结果表明，灵活地预训练单个 ViT 可能比预训练多个固定 ViT 更为可取。

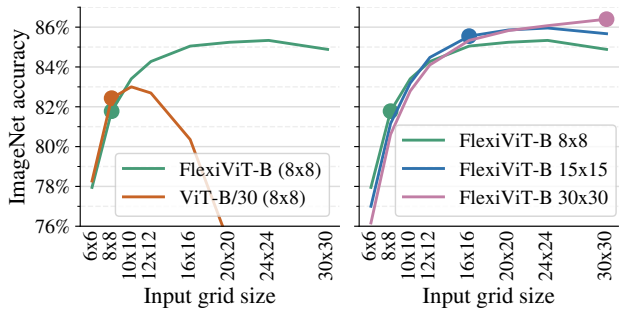

图 8. **快速迁移**。 FlexiViT 可以在较小的序列长度下进行低成本微调，并在测试时使用更长的序列以实现更高的性能。(左) FlexiViT-B 和 ViT-B/30 模型在 8x8 网格大小（由点表示）下进行微调，并在其他网格大小下进行评估。标准 ViT 模型的准确率迅速下降，而 FlexiViT 在网格大小增加时表现出显著的性能提升。(右) 单个 FlexiViT-B 模型在三种不同的网格大小（由点表示）下进行微调，并在各种网格大小下进行评估。

我们通过将 FlexiViT-B 模型（在 ImageNet-21k 上使用蒸馏进行预训练）迁移到 ImageNet-1k 数据集来进行实验，并使用类似预训练的固定 ViT-B/30 模型作为基准。预训练的 FlexiViT 在较大的网格大小下仍然表现良好，即使在固定大小迁移后也是如此。例如，我们可以在 8×8 网格大小下进行相对廉价的微调。当在 8×8 网格大小下评估时，模型达到了 81.8% 的准确率，但在 24×24 网格大小下评估时，它达到了 85.3% 的 top-1 准确率，在没有额外训练成本的情况下获得了 3.5% 的准确率提升（图 8）。有关微调设置的更多详细信息可以在附录 D 中找到。

5. 灵活化现有训练设置

迄今为止，我们主要关注在预训练阶段使模型具备灵活性。现在，我们将展示现有的预训练模型在迁移至下游任务时同样能够实现灵活化。接下来，我们将对一系列现有的训练设置进行灵活化处理。

5.1. 迁移学习

我们使用与第 4 节相同的 6 个迁移数据集，并采用相同的设置。我们再次在图 9 中展示了 SUN397 的结果，并在附录 E 中展示了所有其他数据集的结果。不同之处在于，我们现在在迁移过程中随机化补丁大小，并在不同补丁大小（x 轴，三组柱状图）下评估单一生成的模型。

FlexiViT 的灵活迁移效果最佳，但即便在迁移过程中对固定模型进行灵活化处理，考虑到迁移所用的极短训练时间和低学习率，其表现也出人意料地好。图中以一条短横线标示了在固定 patch 大小下迁移并评估的固定大小模型的基准表现。

5.2. 多模态图文训练

接下来，我们讨论两种灵活化多模态图文训练的方法：FlexiLiT 和 FlexiCLIP。在 FlexiLiT 中，我们训练一个文本塔以生成与来自不同 patch 大小的视觉嵌入对齐良好的文本嵌入（B/flexi）。提供了直接使用固定分辨率的 FlexiViT 模型或 ViT 模型的 LiT 基准。图 10 展示了在 Flickr30k [45] 数据集上的 zero-shot 图像到文本检索结果。FlexiLiT-B/flexi 平均表现最佳，而使用 FlexiViT-B/30 和 FlexiViT-B/16 的 LiT 都获得

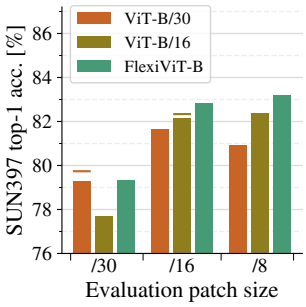

图 9. 柔性模型的灵活迁移效果最佳，但即便是刚性模型，在迁移过程中也能实现灵活化。图中线条代表固定模型的直接迁移，以供参考。

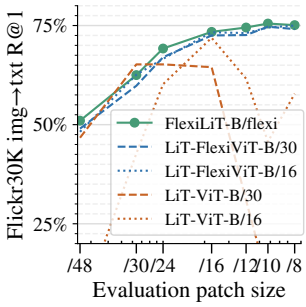

图 10. Flexified LiT 迁移效果最佳，而使用单一 patch Size（30 或 16）的 FlexiViT 固定迁移表现也出人意料地好，与第 4.2 节类似。

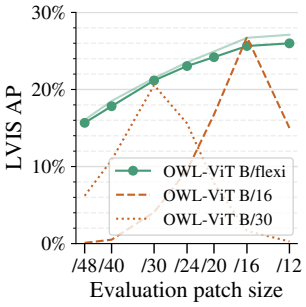

图 11. 灵活化的开放词汇检测。图像文本模型被迁移到检测任务中 [40]。深绿色表示固定模型的迁移，浅绿色表示灵活模型的迁移。

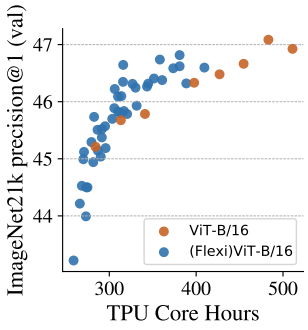

图 12. 灵活调整补丁大小以加速预训练。ViT-B/16 模型在不同轮次下训练，(Flexi)ViT-B/16 采用不同的课程安排。评估时补丁大小为 16。

了非常接近的结果。灵活化还提供了快速迁移的可能性，如第 4.2 节所讨论。图 10 中展示的 LiT-ViT 基准在训练时所使用的序列长度上与 FlexiLiT 相匹配，但在推理时使用不同序列长度时性能迅速下降。我们在从头开始的图文训练设置（即 FlexiCLIP）中观察到了类似的结论（更多结果见附录 G）。

5.3. 开放词汇检测

在图像级别任务之外，我们发现灵活化同样适用于目标检测训练。我们修改了 OWL-ViT 的训练过程，引入了如算法 1所述的灵活补丁大小。与分类类似，灵活的 OWL-ViT 检测模型在推理时，无论补丁大小如何，其表现均接近或优于固定大小的模型（图 11）。此外，我们发现对于检测任务，最优补丁大小并不一定是最小的。在 35 个检测数据集 [34] 上的评估表明，推理时调整补丁大小相比在最小补丁大小下评估能带来更好的结果（附录 E）。这使得灵活化对于检测尤为有价值。

5.4. 训练时间与灵活性

除了拥有灵活的模型外，还可以利用 FlexiViT 的机制来更快地预训练固定大小的 ViT。在这种情况下，我们指定一个课程：一个关于 patch 大小的概率分布序列 $(p_k)_{k=1}^K$，以及一个映射 $c : \mathbb{N} \to [K]$，用于确定在训练步骤 t 时使用哪个分布 p_k。例如，如果所需的 patch 大小为 16×16，则序列 p_K 中的最后一个概率分布将全部集中在该 patch 大小上。可以设计多种课程，详见附录 H。在图 12中，我们展示了随着训练长度的变化，使用 patch 大小课程进行训练通常比标准训练在计算预算内能获得更好的性能。

6. 分析 FlexiViTs

我们发现，减小 patch 的

Attention relevance patterns across scales　大小会导致 [9] 注意力相关性集中在图像中更多较小的区域。在图 13（顶部）中，我们观察到注意力在不同尺度下会发生显著变化。

Relation of token representations across scales　当我们减小 FlexiViT 的块大小时，每个 token “分裂” 成多个 token。一个自然的问题是较大块大小的 token 表示与较小块大小的 token 表示之间有什么关系。为了解答这个问题，我们测量了在一种块大小的特征图中心的 “种子” token 的表示与其他在相同和不同块大小下的 token 的表示之间的余弦相似度。如图 13（底部）所示，我们确实能够在不同尺度的 token 之间找到对应关系。

Ensembling　我们探讨了通过在多个尺度上对相同 FlexiViT 的预测进行集成，是否可以提高预测准确率。我们发现，从总计算量来看，运行一个单一的 FlexiViT 比集成多个较小的模型通常要好得多。完整结果见附录 J。

Shape or texture bias　ViT 倾向于使用形状或纹理特

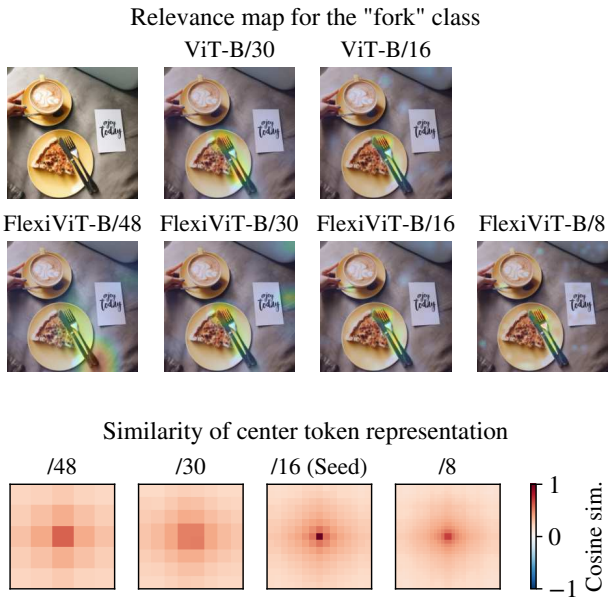

图 13. **跨尺度分析 FlexiViT 的注意力和 token 表示**。上图：注意力相关性（如 [9] 所示）在不同 patch 大小下可能显著变化。例如，FlexiViT-B/48 和 FlexiViT-B/8 认为输入中对于类别“叉子”最相关的区域不同。更多示例见附录 L。下图：FlexiViT-B 在 patch 大小为 16 时，特征图中心种子 token 表示与其他 patch 大小下 token 表示之间的余弦相似度。表示取自第 6 块，并在我们的 I21K 验证集上取平均。其他块和 patch 大小的类似图见附录 I。

征的偏差 [18] 已被证明在很大程度上取决于其补丁大小 [6]。在附录 K 中，我们展示了在每个补丁大小下评估的 FlexiViT 与在同一补丁大小下训练和评估的 ViT 具有相似的纹理偏差。

Model and dataset size　在整篇论文中，我们专注于在 12 M 张图像上训练的 base 大小（-B）的 FlexiViT 模型。为了验证这两种情景都不是必需的，我们使用 ImageNet-1k DeiT III 模型 [58] 作为教师，在 ImageNet-1k（1.2 M 张图像）上训练 FlexiViT-S,B,L 模型。我们可以看到在图 2中，一个单独的 FlexiViT-L 模型可以与所有三个 DeiT III 模型和 EfficientNetV2 相媲美或表现更好。然而，仍然存在一个点，在该点改变模型宽度比改变块大小更有效。ImageNet-ReaL/v2/A/R [3, 22, 23, 48] 的数值结果和评估见附录 F，使用 GFLOPs [15] 的图 2的版本见附录 P。

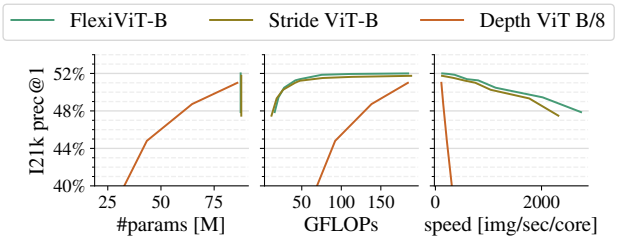

图 14. **调整步幅与深度作为 FlexiViT 的替代方案**

7. 替代方案讨论

改变输入补丁的大小并非 ViTs 中平衡序列长度与计算量的唯一途径。我们在核心设置中探索了两种替代方案：在 ImageNet-21k 上进行蒸馏。

Varying patch embedding stride　另一种方法是固定块大小并改变其采样步幅，即提取重叠块以增加序列长度。直觉上，这种方法的优势在于固有的块大小是固定的，并且在计算块嵌入时我们避免了任何特殊处理。图 14 的结果表明，改变步幅的效果几乎一样好，仅略微落后于我们的基准。

Varying model depth　另一种替代方法是增加深度方面的灵活性，即层数。深度剪枝已在自然语言处理 [17, 50] 的背景下进行了探索，并且最近也用于 ViTs [66]。深度剪枝与 FlexiViT 有根本的不同：它在深度上呈线性扩展，使用参数的一个子集，并允许逐步改进预测。我们通过将共享头连接到不同的中间层来随机化深度。我们还尝试了分离的头，但效果更差。图 14中的结果表明，FlexiViT 在计算-准确率权衡方面明显优于深度剪枝。

8. 结论

FlexiViT 是一种简单高效的方法，通过单一模型在计算与预测性能之间实现平衡，这得益于 ViTs 独特的补丁嵌入策略。FlexiViT 能够显著降低预训练成本，因为它只需一次性训练一个模型即可适应所有规模，并在多种下游任务中表现出色。未来研究有许多令人兴奋的方向，我们期待我们的成果能激励社区探索更多补丁化应用的创新可能。

9. 致谢

我们感谢 Daniel Keysers 对论文初稿的宝贵反馈，感谢 Geoffrey Hinton 在项目初期给予的推动，同时也感谢各自在谷歌的团队对鼓励创新与独立研究的大力支持。

我们还要感谢以下艺术家通过 unsplash.com 免费提供他们的摄影作品（用于可视化）：Tanya Patrikeyeva、Markus Spiske、Matheus Bardemaker、Alexandru Sofronie、Chris Smith、Kajetan Sumila、Julee Juu、Mike Erskine、Piermanuele Sberni、Feyza Yıldırım 和 Sixteen Miles Out。

我们感谢匿名审稿人提供的宝贵意见，这些反馈进一步提升了我们的论文质量。

最后，图 31 中用作背景的浣熊图片来自 publicdomainvectors.org。

参考文献

[1] Hassan Akbari, Liangzhe Yuan, Rui Qian, Wei-Hong Chuang, Shih-Fu Chang, Yin Cui, and Boqing Gong. VATT: transformers for multimodal self-supervised learning from raw video, audio and text. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, 2021. 1

[2] Lei Jimmy Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. CoRR, abs/1607.06450, 2016. 4, 15

[3] Lucas Beyer, Olivier J. Hénaff, Alexander Kolesnikov, Xiaohua Zhai, and Aäron van den Oord. Are we done with imagenet? CoRR, abs/2006.07159, 2020. 9

[4] Lucas Beyer, Xiaohua Zhai, and Alexander Kolesnikov. Better plain vit baselines for imagenet-1k. CoRR, abs/2205.01580, 2022. 16

[5] Lucas Beyer, Xiaohua Zhai, Amélie Royer, Larisa Markeeva, Rohan Anil, and Alexander Kolesnikov. Knowledge distillation: A good teacher is patient and consistent. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 10925–10934, 2022. 2, 5, 22

[6] Srinadh Bhojanapalli, Ayan Chakrabarti, Daniel Glasner, Daliang Li, Thomas Unterthiner, and Andreas Veit. Understanding robustness of transformers for image classification. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 10231–10241, 2021. 8

[7] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101–mining discriminative components with random forests. In European conference on computer vision, pages 446–461. Springer, 2014. 6

[8] Han Cai, Chuang Gan, and Song Han. Once for all: Train one network and specialize it for efficient deployment. CoRR, abs/1908.09791, 2019. 2

[9] Hila Chefer, Shir Gur, and Lior Wolf. Transformer interpretability beyond attention visualization. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 782–791, June 2021. 8, 9, 21, 25

[10] Wuyang Chen, Wei Huang, Xianzhi Du, Xiaodan Song, Zhangyang Wang, and Denny Zhou. Auto-scaling vision transformers without training. In The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022. OpenReview.net, 2022. 1, 2

[11] Xinlei Chen, Hao Fang, Tsung-Yi Lin, Ramakrishna Vedantam, Saurabh Gupta, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO captions: Data collection and evaluation server. CoRR, abs/1504.00325, 2015. 17, 18

[12] Jang Hyun Cho and Bharath Hariharan. On the efficacy of knowledge distillation. In Proceedings of the IEEE/CVF international conference on computer vision, pages 4794–4802, 2019. 5

[13] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. CoRR, abs/1604.01685, 2016. 6

[14] Corinna Cortes, Mehryar Mohri, and Afshin Rostamizadeh. Algorithms for learning kernels based on centered alignment. The Journal of Machine Learning Research, 13:795–828, 2012. 5

[15] Mostafa Dehghani, Anurag Arnab, Lucas Beyer, Ashish Vaswani, and Yi Tay. The efficiency misnomer. CoRR, abs/2110.12894, 2021. 9, 23

[16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In International Conference on Learning Representations, 2021. 1, 3, 6, 16, 17

[17] Angela Fan, Edouard Grave, and Armand Joulin. Reducing transformer depth on demand with structured dropout. In 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020. OpenReview.net, 2020. 9

[18] Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A Wichmann, and Wieland Brendel. Imagenet-trained cnns are biased towards texture; increasing shape bias improves accuracy and robustness. arXiv preprint arXiv:1811.12231, 2018. 8, 21

[19] Chengyue Gong, Dilin Wang, Meng Li, Xinlei Chen, Zhicheng Yan, Yuandong Tian, Qiang Liu, and Vikas Chandra. Nasvit: Neural architecture search for efficient vision transformers with gradient conflict aware supernet training. In The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022. OpenReview.net, 2022. 2

[20] Agrim Gupta, Piotr Dollar, and Ross Girshick. LVIS: A dataset for large vocabulary instance segmentation. In CVPR, June 2019. 6, 18

[21] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross B. Girshick. Masked autoencoders are scalable vision learners. In IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022, pages 15979–15988. IEEE, 2022. 1, 2

[22] Dan Hendrycks, Steven Basart, Norman Mu, Saurav Kadavath, Frank Wang, Evan Dorundo, Rahul Desai, Tyler Zhu, Samyak Parajuli, Mike Guo, Dawn Song, Jacob Steinhardt, and Justin Gilmer. The many faces of robustness: A critical analysis of out-of-distribution generalization. In 2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021, pages 8320–8329. IEEE, 2021. 9

[23] Dan Hendrycks, Kevin Zhao, Steven Basart, Jacob Steinhardt, and Dawn Song. Natural adversarial examples. In IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021, pages 15262–15271. Computer Vision Foundation / IEEE, 2021. 9

[24] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015. 5

[25] Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc V. Le, Yun-Hsuan Sung, Zhen Li, and Tom Duerig. Scaling up visual and vision-language representation learning with noisy text supervision. In Marina Meila and Tong Zhang, editors, Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event, volume 139 of Proceedings of Machine Learning Research, pages 4904–4916. PMLR, 2021. 6

[26] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. Panoptic segmentation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019. 6, 18

[27] Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Joan Puigcerver, Jessica Yung, Sylvain Gelly, and Neil Houlsby. Large scale learning of general visual representations for transfer. arXiv preprint arXiv:1912.11370, 2(8), 2019. 17

[28] Alexander Kolesnikov, André Susano Pinto, Lucas Beyer, Xiaohua Zhai, Jeremiah Harmsen, and Neil Houlsby. Uvim: A unified modeling approach for vision with learned guiding codes. Advances in Neural Information Processing Systems, 2022. 6, 18

[29] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In International Conference on Machine Learning, pages 3519–3529. PMLR, 2019. 5

[30] Ranjay Krishna, Yuke Zhu, Oliver Groth, Justin Johnson, Kenji Hata, Joshua Kravitz, Stephanie Chen, Yannis Kalantidis, Li-Jia Li, David A Shamma, et al. Visual genome: Connecting language and vision using crowdsourced dense image annotations. International journal of computer vision, 123(1):32–73, 2017. 18

[31] Alex Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, Univ. Toronto, 2009. 6

[32] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham M. Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representations for adaptive deployment. CoRR, abs/2205.13147, 2022. 2

[33] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11):2278–2324, 1998. 1

[34] Chunyuan Li, Haotian Liu, Liunian Harold Li, Pengchuan Zhang, Jyoti Aneja, Jianwei Yang, Ping Jin, Houdong Hu, Zicheng Liu, Yong Jae Lee, and Jianfeng Gao. ELEVATER: A Benchmark and Toolkit for Evaluating Language-Augmented Visual Models. In NeurIPS, 2022. 8, 22, 24

[35] Chunyuan Li, Jianwei Yang, Pengchuan Zhang, Mei Gao, Bin Xiao, Xiyang Dai, Lu Yuan, and Jianfeng Gao. Efficient self-supervised vision transformers for representation learning. CoRR, abs/2106.09785, 2021. 2

[36] Mingbao Lin, Mengzhao Chen, Yuxin Zhang, Ke Li, Yunhang Shen, Chunhua Shen, and Rongrong Ji. Super vision transformer. CoRR, abs/2205.11397, 2022. 2

[37] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In European conference on computer vision, 2014. 6

[38] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, Furu Wei, and Baining Guo. Swin Transformer V2: Scaling Up Capacity and Resolution. CoRR, abs/2111.09883, 2021. 2

[39] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 11976–11986, 2022. 3

[40] Matthias Minderer, Alexey Gritsenko, Austin Stone, Maxim Neumann, Dirk Weissenborn, Alexey Dosovitskiy, Aravindh Mahendran, Anurag Arnab, Mostafa Dehghani, Zhuoran Shen, Xiao Wang, Xiaohua Zhai, Thomas Kipf, and Neil Houlsby. Simple open-vocabulary object detection with vision transformers. In European Conference on Computer Vision. ECCV, 2022. 6, 8, 18, 22

[41] Basil Mustafa, Carlos Riquelme, Joan Puigcerver, Rodolphe Jenatton, and Neil Houlsby. Multimodal contrastive learning with limoe: the language-image mixture of experts. arXiv preprint arXiv:2206.02770, 2022. 1

[42] Thao Nguyen, Maithra Raghu, and Simon Kornblith. Do wide and deep networks learn the same things? uncovering how neural network representations vary with width and depth. In International Conference on Learning Representations, 2021. 5

[43] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In 2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing, pages 722–729. IEEE, 2008. 6

[44] Omkar M Parkhi, Andrea Vedaldi, Andrew Zisserman, and CV Jawahar. Cats and dogs. In 2012 IEEE conference on computer vision and pattern recognition, pages 3498–3505. IEEE, 2012. 6

[45] Bryan A. Plummer, Liwei Wang, Chris M. Cervantes, Juan C. Caicedo, Julia Hockenmaier, and Svetlana Lazebnik. Flickr30k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models. In 2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015, pages 2641–2649. IEEE Computer Society, 2015. 7, 17

[46] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In Marina Meila and Tong Zhang, editors, Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event, volume 139 of Proceedings of Machine Learning Research, pages 8748–8763. PMLR, 2021. 6, 18

[47] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: Ef-

ficient vision transformers with dynamic token sparsification. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, pages 13937–13949, 2021. 1, 2

[48] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do imagenet classifiers generalize to imagenet? In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA, volume 97 of Proceedings of Machine Learning Research, pages 5389–5400. PMLR, 2019. 9

[49] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. International journal of computer vision, 115(3):211–252, 2015. 3, 17

[50] Tal Schuster, Adam Fisch, Tommi Jaakkola, and Regina Barzilay. Consistent accelerated inference via confident adaptive transformers. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pages 4962–4979, 2021. 9

[51] Shuai Shao, Zeming Li, Tianyuan Zhang, Chao Peng, Gang Yu, Xiangyu Zhang, Jing Li, and Jian Sun. Objects365: A Large-Scale, High-Quality Dataset for Object Detection. In ICCV, pages 8429–8438, 2019. 18

[52] Samuel Stanton, Pavel Izmailov, Polina Kirichenko, Alexander A Alemi, and Andrew G Wilson. Does knowledge distillation really work? Advances in Neural Information Processing Systems, 34:6906–6919, 2021. 5

[53] Andreas Peter Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers. Transactions on Machine Learning Research, 2022. 3, 4, 5, 15, 22

[54] Robin Strudel, Ricardo Garcia, Ivan Laptev, and Cordelia Schmid. Segmenter: Transformer for semantic segmentation. In ICCV, 2021. 6, 18

[55] Mingxing Tan and Quoc V. Le. Efficientnetv2: Smaller models and faster training. In Marina Meila and Tong Zhang, editors, Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event, volume 139 of Proceedings of Machine Learning Research, pages 10096–10106. PMLR, 2021. 2

[56] Yehui Tang, Kai Han, Yunhe Wang, Chang Xu, Jianyuan Guo, Chao Xu, and Dacheng Tao. Patch slimming for efficient vision transformers. In IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022, pages 12155–12164. IEEE, 2022. 1, 2

[57] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Herve Jegou. Training data-efficient image transformers & distillation through attention. In International Conference on Machine Learning, volume 139, pages 10347–10357, July 2021. 1, 5

[58] Hugo Touvron, Matthieu Cord, and Hervé Jégou. Deit III: revenge of the vit. CoRR, abs/2204.07118, 2022. 1, 2, 9

[59] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021. 1

[60] Yulin Wang, Rui Huang, Shiji Song, Zeyi Huang, and Gao Huang. Not all images are worth 16x16 words: Dynamic transformers for efficient image recognition. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, pages 11960–11973, 2021. 2

[61] Alex H Williams, Erin Kunz, Simon Kornblith, and Scott Linderman. Generalized shape metrics on neural representations. Advances in Neural Information Processing Systems, 34:4738–4750, 2021. 5

[62] Chao-Yuan Wu, Ross B. Girshick, Kaiming He, Christoph Feichtenhofer, and Philipp Krähenbühl. A

multigrid method for efficiently training video models. CoRR, abs/1912.00998, 2019. 2

[63] Jianxiong Xiao, James Hays, Krista A Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In 2010 IEEE computer society conference on computer vision and pattern recognition, pages 3485–3492. IEEE, 2010. 6

[64] Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V Le. Self-training with noisy student improves imagenet classification. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 10687–10698, 2020. 5

[65] Hongxu Yin, Arash Vahdat, Jose M. Alvarez, Arun Mallya, Jan Kautz, and Pavlo Molchanov. A-vit: Adaptive tokens for efficient vision transformer. In IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022, pages 10799–10808. IEEE, 2022. 1, 2

[66] Fang Yu, Kun Huang, Meng Wang, Yuan Cheng, Wei Chu, and Li Cui. Width & depth pruning for vision transformers. In Thirty-Sixth AAAI Conference on Artificial Intelligence, AAAI 2022, Thirty-Fourth Conference on Innovative Applications of Artificial Intelligence, IAAI 2022, The Twelveth Symposium on Educational Advances in Artificial Intelligence, EAAI 2022 Virtual Event, February 22 - March 1, 2022, pages 3143–3151. AAAI Press, 2022. 9

[67] Jiahui Yu, Pengchong Jin, Hanxiao Liu, Gabriel Bender, Pieter-Jan Kindermans, Mingxing Tan, Thomas S. Huang, Xiaodan Song, Ruoming Pang, and Quoc Le. Bignas: Scaling up neural architecture search with big single-stage models. In Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm, editors, Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part VII, volume 12352 of Lecture Notes in Computer Science, pages 702–717. Springer, 2020. 2

[68] Rowan Zellers, Jiasen Lu, Ximing Lu, Youngjae Yu, Yanpeng Zhao, Mohammadreza Salehi, Aditya Kusupati, Jack Hessel, Ali Farhadi, and Yejin Choi. MERLOT RESERVE: neural script knowledge through vision and language and sound. In IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022, pages 16354–16366. IEEE, 2022. 1

[69] Xiaohua Zhai, Alexander Kolesnikov, Neil Houlsby, and Lucas Beyer. Scaling vision transformers. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 12104–12113, 2022. 1, 18

[70] Xiaohua Zhai, Xiao Wang, Basil Mustafa, Andreas Steiner, Daniel Keysers, Alexander Kolesnikov, and Lucas Beyer. Lit: Zero-shot transfer with locked-image text tuning. In IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022, pages 18102–18112. IEEE, 2022. 6, 17, 18, 22

[71] Bolei Zhou, Hang Zhao, Xavier Puig, Tete Xiao, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Semantic understanding of scenes through the ADE20K dataset. Int. J. Comput. Vis., 127(3):302–321, 2019. 6

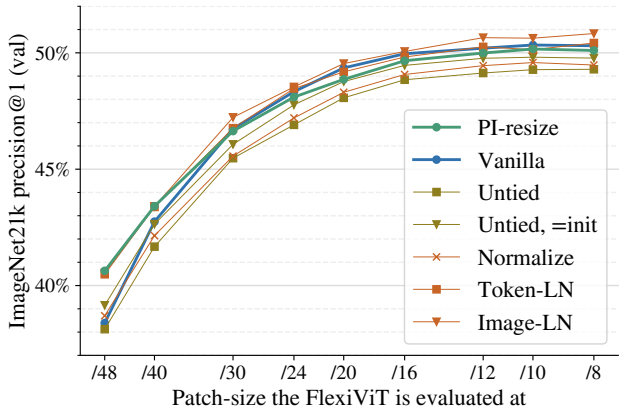

图 15. **处理** patch-Embedding **的选项**。在 ImageNet-21k 上进行从头开始的 Supervised Training。Vanilla（双线性）是最简单的方法且效果良好。PI-resize 进一步改善了大 patch 的情况，并提供了其他优势（见正文）。Untied 意味着为每个 Size 学习 Separate 的 patch-Embedding Kernels，但效果并不理想。Normalize、Token-LN 和 Image-LN 几乎都需要对 Model 进行修改，使其与标准 ViT 不兼容。

除了在正文中提到的各个部分提供更多细节和结果外，我们还在附录 R 中提供了所有图形的完整数值（表格）结果，以便于未来工作的复现/比较。

最后，关于第 7 节中讨论的灵活性替代方法的更多细节，请参见附录 O。

A. 关于灵活补丁 Size 的更多详情

在本节中，我们进一步详细阐述了灵活 Size 的诸多细节。当不关心保留 Model 架构时，我们在附录 A.1中提供了处理灵活 Size 的替代方法的结果。我们在附录 A.2中详细推导了 PI-resize，并在附录 A.3中展示了一些 PI-resize 的 Matrices。此外，我们在附录 A.4中展示了一些 patch-Embedding 权重的可视化结果，包括原始和调整 Size 后的情况。

A.1. 处理灵活补丁大小的替代方案

除了双线性调整（称为 Vanilla）或 PI-resizing 来调整 patch-embedding 权重以处理可变 patch 大小之外，这里我们还讨论并比较了其他几种替代方案。

Untied 为每个大小分配权重，即为每个补丁大小拥有分离的可训练参数缓冲区。

Untied, =init 与上述相同，但将所有补丁嵌入权重初始化为与参考初始化相同的（PI 调整后的）值。在此情景下，模型在初始化时仍与标准 ViT 模型兼容，然而，这些参数在训练过程中可能且确实会发散，从而形成一种非标准的 ViT 架构。

Normalize 简单地对由 patch-embedding 计算得到的 tokens 进行逐个 l2 归一化，使其成为单位范数。这种方法以简单且无需参数的方式解决了所有与范数相关的问题，但与预训练的标准 ViT 模型不兼容。

Token-LN and Image-LN 在 patch-embedding 后添加一个 LayerNorm [2]，它们仅在执行规范化的轴上有所不同。这同样解决了与范数相关的问题，但引入了可学习的参数，并且与预训练的标准 ViT 模型不兼容。

在遵循 [53] 的标签监督训练设置下，对 ImageNet-21k 上的所有变体进行了比较，但训练了 90 轮次。图 15中展示的结果表明，简单的调整大小和 PI 调整大小是最佳解决方案之一，并且还具有生成标准 ViT 模型的额外优势。此外，图中未显示的是，两种 Untied 变体在训练的前半段表现出略微不稳定的训练曲线，而所有其他变体则训练平稳。

A.2. PI-大小调整推导

我们可以将式 (2) 中的目标函数重写如下：

$$\begin{aligned}
&\mathbb{E}_{x\sim\mathcal{X}}\left[(\langle x,\omega\rangle-\langle Bx,\hat{\omega}\rangle)^2\right]= \\
&\mathbb{E}_{x\sim\mathcal{X}}\left[(x^T(\omega-B^T\hat{\omega}))^2\right]= \\
&\mathbb{E}_{x\sim\mathcal{X}}\left[((\omega-B^T\hat{\omega})^Tx)(x^T(\omega-B^T\hat{\omega}))\right]= \\
&(\omega-B^T\hat{\omega})^T\mathbb{E}_{x\sim\mathcal{X}}\left[xx^T\right](\omega-B^T\hat{\omega})= \\
&\|\omega-B^T\hat{\omega}\|^2_\Sigma,
\end{aligned} \tag{6}$$

其中 $\|v\|^2_\Sigma=v^T\Sigma v$ 和 $\Sigma=\mathbb{E}_{x\sim\mathcal{X}}xx^T$ 是 $\mathcal{X}$ 的（未中心化）协方差矩阵。当 $\mathcal{X}=\mathcal{N}(0,I)$ 时，我们恢复标准的欧几里得范数 $\|\omega-B^T\hat{\omega}\|^2$。

最后，我们注意到伪逆矩阵为线性方程组提供了一个最小二乘解：

$$(B^T)^+\omega\in\arg\min_{\hat{\omega}}\|\omega-B^T\hat{\omega}\|^2. \tag{7}$$

我们还可以为任意 $\Sigma=\mathbb{E}_{x\sim\mathcal{X}}xx^T$ 推导出一个解析解。注意 $\|v\|^2_\Sigma=v^T\Sigma v=(\sqrt{\Sigma}v)^T\sqrt{\Sigma}v=\|\sqrt{\Sigma}v\|^2$ 。于是，我们得到

$$\|\omega-B^T\hat{\omega}\|^2_\Sigma=\|\sqrt{\Sigma}\omega-\sqrt{\Sigma}B^T\hat{\omega}\|^2. \tag{8}$$

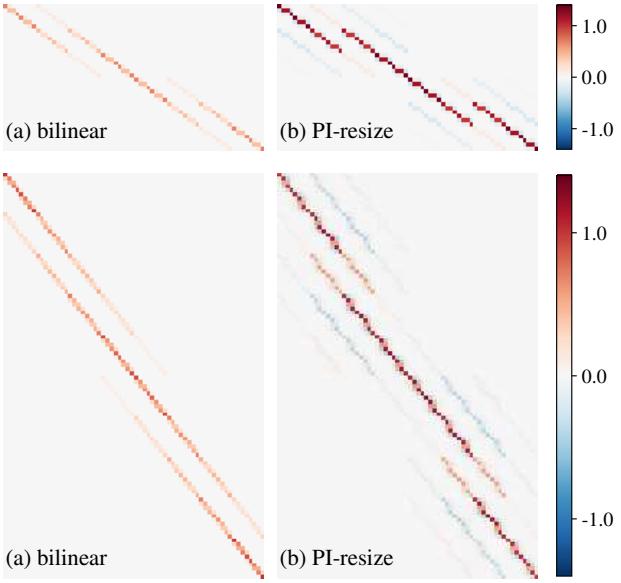

图 16. **调整大小矩阵的可视化**。每一行对应于计算一个输出像素时使用的权重。非对角线元素对应于当前像素下方/上方的像素。我们可以看到，PI-resize 确实包含负权重，具有更大的感受野，并且总体上使用比双线性调整更大的权重。

最优解随后得出

$$(\sqrt{\Sigma}B^T)^+\sqrt{\Sigma}\omega \in \arg\min_{\hat{\omega}} \|\omega - B^T\hat{\omega}\|^2. \qquad (9)$$

A.3. 部分 PI 调整矩阵的可视化

我们在图 16中可视化了双线性和 PI-resize 操作的上采样和下采样矩阵，以便进行视觉比较。

A.4. 可视化补丁嵌入权重

Figs. 33 to 37 中的补丁嵌入的 PCA（参见 [16]）。

B. “潜在”参数形态

FlexiViT 确实引入了两个在原始 ViT 架构中不存在的新超参数：潜在的 patch-embedding 权重和位置嵌入的大小（即在调整大小之前的学成参数）。

然而，只要这两个参数处于“合理”值域内，它们（或许令人惊讶地）对模型最终性能的影响微乎其微。我们在两种不同情景下验证了这一点。

For the patch-size parameter，因为它受到所使用的调整方法的影响，我们在完整的 FlexiViT 设置中进行消融实验。图 17 (b) 显示在所有评估大小之间没有显著差异，因此我们坚持最初任意的默认值 32 进行所有实验。

For the position embedding parameter，我们进行了一项早期实验，使用从头开始训练的 ViT-S/16 模型在 ImageNet-1k 数据集上进行了训练，遵循了 [4] 的方法。图 17 (c) 显示，即使对于普通的 ViT 训练，这种方法也是可行的，并且训练曲线基本上不受图内双线性调

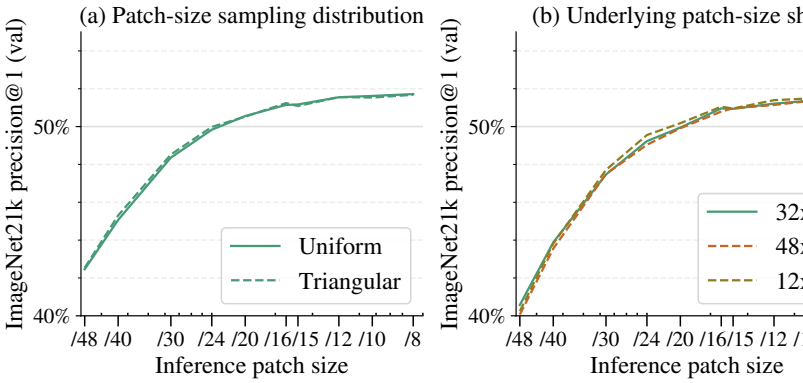

图 17. **消融一些不重要的超参数，**因为改变它们的值在结果中并未显示出显著
因其简单性而更受青睐，但出于历史原因，我们有时会使用“三角”分布（对中
丁赋予较少权重）。(b) 在全套 FlexiViT 训练设置中，调整潜在补丁嵌入参数的
基准模型潜在位置嵌入的形状，遵循 [4]。

整位置嵌入的影响。

C. 斑块大小采样分布

在本项目的大约前半段，我们从一种非均匀的分布中采样了样本大小，该分布使得 16 到 30 之间的样本大小被采样的频率是此范围外的样本大小的三倍。这种“三角”分布完全基于直觉，而一旦我们验证了它并不比均匀采样更优（实验如图 17 (a) 所示），为了简化，我们决定采用均匀分布。进一步地，我们决定避免重新运行迄今为止的所有实验所带来的高昂成本，因此，主论文中的一些实验仍使用了“三角”分布。然而，在整篇论文中所有直接比较的曲线，都是在完全相同的条件下进行训练的。

D. 资源高效转移的详细信息

在 ImageNet-1k 数据集上微调 FlexiViT 模型时，我们通常遵循 [16] 中的迁移学习设置。我们采用 SGD 动量优化器，初始学习率为 0.03，并应用余弦学习率衰减。此外，我们将预训练参数的学习率降低 10 倍。我们使用批量大小为 512 和输入图像尺寸为 480×480，通过 Inception 裁剪和左右翻转增强，进行 20000 步优化。

E. 使用预训练的 FlexiViT 模型：更多细节与成果

在本节中，我们提供了第 4 节所述场景的更多细节和完整结果：使用预训练的 FlexiViT 模型。

有关第 5 节中讨论的灵活化训练流程的更多详情，我们建议参阅附录 M 了解灵活化迁移学习，附录 G 了解灵活化对比图文学习（LiT 和 CLIP），以及附录 N 了解灵活化开放词表检测（OWL-ViT），包括在 ELEVATER 上的结果。

E.1. 使用 FlexiViT 模型进行迁移

在迁移过程中，我们遵循了简单的 BiT-HyperRule [27]。简而言之，我们进行了相对较短的迁移步骤（对于 flowers 和 pets 为 500 步，对于 food101 和 sun 为 2500 步，对于 CIFAR 为 10 000 步），使用带有 0.9 动量的 SGD 优化器，没有权重衰减，没有暂退法 (Dropout)，除了翻转和随机裁剪外没有其他数据增强。我们将新的分类层初始化为全零，并采用了短期的学习率预热，这两者都有助于更好地保留预训练的权重。唯一与 [27] 不同的情景是，我们确实针对每个任务单独在 $\{0.03, 0.01, 0.003, 0.001\}$ 范围内扫描了学习率。我们在图 18中展示了所有使用的 6 个数据集的结果。

E.2. 在 LiT 中使用 FlexiViT 模型进行图像-文本任务

我们使用与 [70] 中相同的 4B 图像-文本对数据集来训练 LiT 模型，并采用与 LiT 模型相同的超参数。唯一的不同在于此处使用了 FlexiViT 模型，而非标准的 ViT 模型。FlexiViT 模型在固定序列长度下进行迁移，即 30^2 或 16^2，并采用 240×240 的图像分辨率。我们在 ImageNet [49] 上报告了 zero-shot 分类结果，在 MS-COCO [11] 和 Flickr30K [45] 上报告了 zero-shot 图像到文本/文本到图像检索结果，如图 19所示。

E.3. 在 OWL-ViT 中使用 FlexiViT 模型进行 zero-shot 检测

为了评估 FlexiViT 骨干网络在开放词汇检测中的表现（图 7），我们比较了使用固定或灵活 LiT-B 模型初始化的 OWL-ViT。具体而言，这些骨干网络以固定或灵活预训练的 ViT 图像模型为起点，随后冻结并在

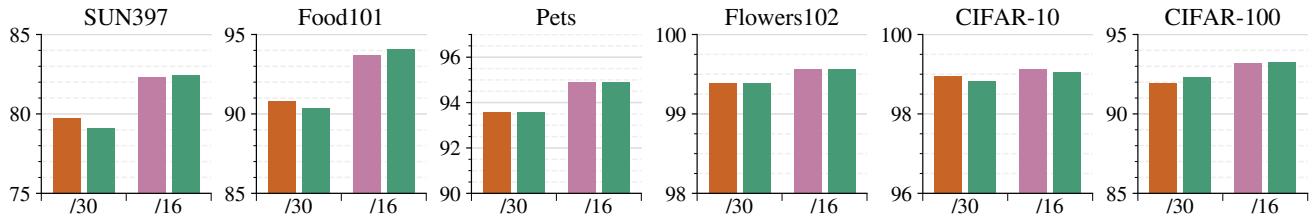

图 18. **更多迁移结果**。图例与图 7 相同。主要结论是，在广泛的图像分类任务中，结果在质量上是相同的，即使仅在迁移后使用固定的补丁大小，也可以安全地使用预训练的 FlexiViT 模型代替 ViT 模型。

固定 patch 大小下与文本模型进行 LiT 微调 [70]（与最终评估相同，即分别为 30^2 和 16^2）。使用这些骨干网络的 OWL-ViT 在固定 patch 大小（30^2 或 16^2）下，以 720×720 的分辨率在 Objects365 [51] 和 Visual Genome [30] 上进行训练，如原论文 [40] 所述。我们在 LVIS [20] 上报告了平均准确率（AP）。

E.4. 在 UViM 中使用 FlexiViT 模型进行全景分割

除了使用 FlexiViT 模型权重进行初始化外，我们尽可能接近原始 UViM 设置 [28]。特别是，我们在 COCO 全景数据集 [11] 上训练模型，并在官方验证分割上报告标准的 PQ 指标 [26]。我们使用自定义的 adafactor 优化器变体 [69]，基础学习率为 0.001 对模型预训练部分的学习率降低了一个因子 10。输入图像大小为 512×512 。更多关于训练设置的详细信息可以在 UViM 论文 [28] 和 UViM 模型的官方仓库中找到。[6].

E.5. 在 Segmenter 中使用 FlexiViT 模型进行语义分割

我们遵循 Segmenter [54] 的实验设置，对带有线性解码器的 Vision Transformer 进行端到端微调。在训练期间，我们采用数据增强策略，包括随机调整图像大小（比例在 0.5 到 2.0 之间）、光度增强以及随机水平翻转。我们通过填充随机裁剪图像至 480×480 分辨率，从而保持宽高比不变。对于 Cityscapes 和 ADE20k 数据集，均采用 480×480 分辨率进行训练。训练过程持续 127 个轮次，小批量大小为 16（在 ADE20k 上共计 160k 次迭代）。我们采用“poly”学习率衰减策略，并在所有运行中探索基础学习率于 $\{1e-4, 3e-4, 8e-4\}$ 范围内。权重衰减固定为 0.01。评估时，使用滑动窗口方法，以 480×480 分辨率处理推理过程中不同大小的图像。表 3 第 6 行在 [54] 中报告了 48.06 的 mIoU。在同一情景下，我们的代码库中 6 次运行的平均结果为 47.6 ± 0.4 的 mIoU。ADE20k 上的结果详见表 7。

[6] https://github.com/google-research/big_vision/tree/main/big_vision/configs/proj/uvim

F. 仅基于 ImageNet-1k 的完整数值结果

我们提供了在 ImageNet-1k 上纯训练的 FlexiViTs 的完整数值结果，并在图 2中展示，包括在 Tabs. 1 to 4中的额外鲁棒性和 OOD 测试集上的结果。

G. FlexiLiT 与 FlexiCLIP 成果

FlexiLiT 遵循与第 E.2 节中描述的确切相同的设置，但在 LiT 训练期间随机化 patch 大小。我们在图 20 中展示了更多 FlexiLiT 的结果。

对于 FlexiCLIP，我们只需将 FlexiLiT 中预训练且冻结的骨干网络替换为随机初始化且未冻结的骨干网络，这对应于 [70] 中的 uu 情景，并等同于 CLIP [46]。图 21在此情景下展示了相同的结论。令人欣慰的是，patch 大小的随机化并未阻碍从零开始同时学习图像和文本表示。

H. 加速预训练

如第 5.4 节所述，可以利用 FlexiViT 的方法，通过改变补丁大小并调整嵌入权重来加速 ViT 的预训练。为此，我们定义了一个课程：一个关于补丁大小的概率分布序列 $(p_k)_{k=1}^{K}$，以及一个映射 $c : \mathbb{N} \to [K]$，用于确定在训练步骤 t 时应使用哪个分布 p_k。本节中，我们采用补丁大小序列 $(48, 40, 30, 24, 16)$，以展示计算资源的潜在节省。

我们将 16×16 设置为评估期间所需的/目标补丁大小。因此，我们比较了基于课程学习的 FasterViT 预训练方法与标准的 ViT/B/16 架构，后者在整个训练

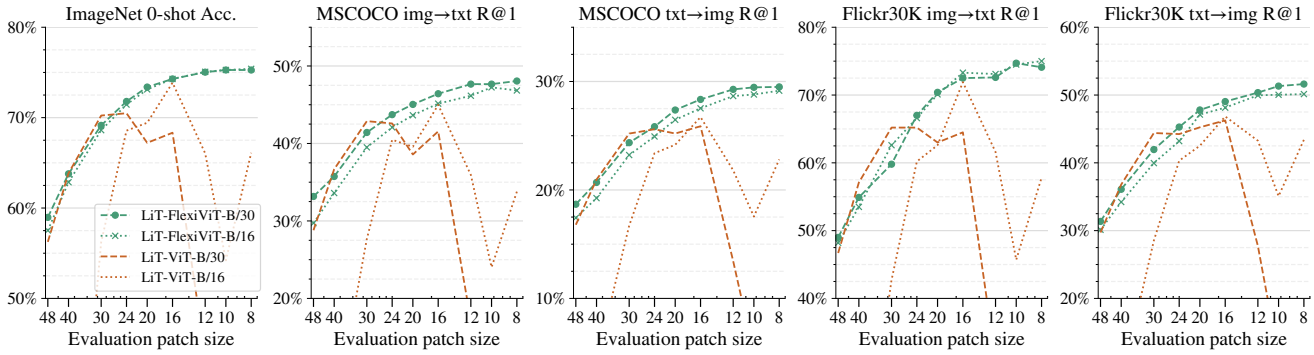

图 19. **固定补丁大小下的 FlexiViT 迁移。** LiT-ViT 基准模型在训练时所使用的序列长度（16^2 或 30^2）上与 LiT-FlexiViT 表现相当，但在使用不同推理序列长度时性能迅速下降。我们观察到，尽管在 LiT 迁移过程中仅使用了固定的序列长度，LiT-FlexiViT 模型在不同推理序列长度下均表现良好。这一效应与第 4.2节中描述的情况相似，并在图 20中进一步探讨。

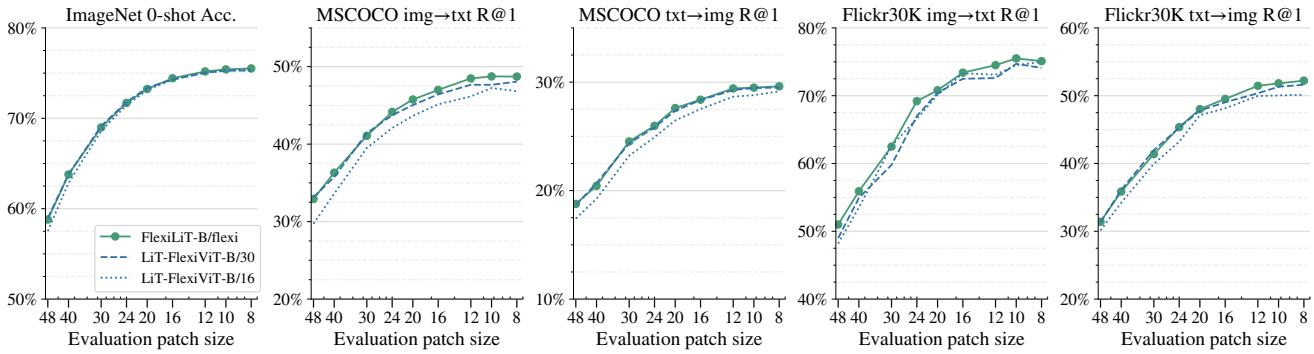

图 20. FlexiLiT. 我们观察到，通过在 LiT 调优期间随机化 patch 大小来灵活化 LiT 训练时，性能有持续但小幅的提升。这再次表明，LiT-FlexiViT 基准表现强劲，并支持快速迁移：使用较大的 patch 大小进行低成本迁移，并以较小的 patch 大小免费提供服务。

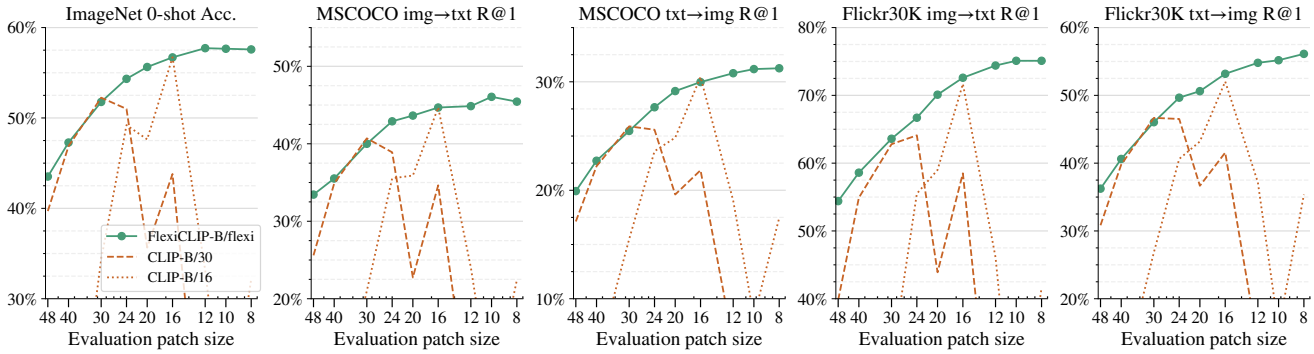

图 21. FlexiCLIP. 我们观察到与 FlexiViT 非常相似的结论，即 FlexiCLIP 模型在推理过程中能够很好地适应多种评估 patch 大小。有趣的是，一个固定序列长度的文本塔能够生成与允许多种序列长度的 FlexiViT 图像塔良好对齐的嵌入。

过程中补丁大小固定为 16×16。除了可变的补丁大小和嵌入层外，两种架构在其他方面完全相同。

在此评估中，我们采用了一个简单的课程学习策略。具体而言，我们设定了一个较大的 patch size（例如 48×48），记作 p_0，以及目标 patch size 16×16，记作 p_1。初始阶段，我们使用较大的 patch size，随后通过 FlexiViT 的 PI-resize 功能切换至较小的 patch size。我们尝试了三种不同的调度方案：$(50\%, 75\%, 90\%)$，其

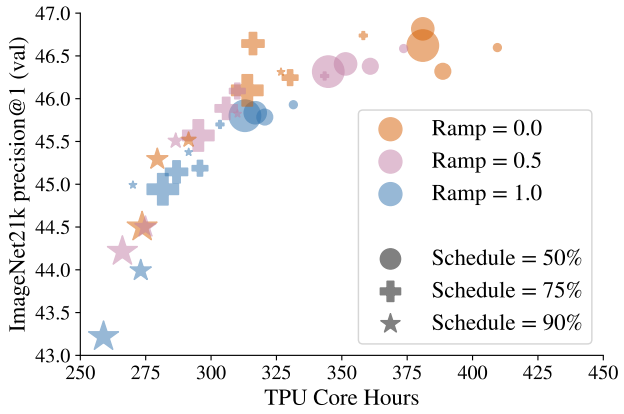

图 22. 详细探讨了更大的补丁大小（标记的大小）、调度和斜坡周期对计算和准确率的影响。详情请参见附录 H。

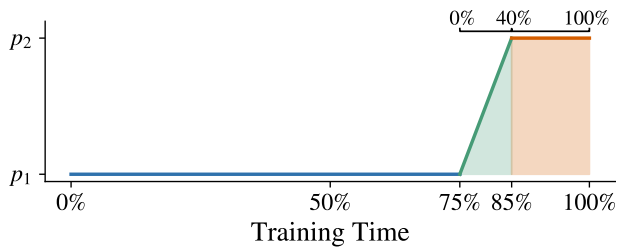

图 23. 该图展示了在加速预训练过程中（参见第 5.4节）和附录 H中，补丁大小分布的变化情况。这里，前 75% 的训练步骤使用较大的补丁大小 p_1，而剩余的 25% 则用于期望/目标补丁大小。图中的过渡期占 40%。

中 schedule = 75% 表示 75% 的训练时间仅使用较大的 patch size。为了避免在 patch size 间直接切换，我们还引入了一个可选的过渡期，如图23所示。例如，40% 的过渡期意味着分配给较小 patch size 的时间中有 40% 用于在两个分布之间逐步过渡。我们试验了三种过渡期：$(0\%, 50\%, 100\%)$。每个实验独立运行，并将计算量与准确率的结果绘制于图12(x) 中。如图所示，FasterViT 在保持与标准 ViTs 相同准确率水平的同时，减少了计算量，尽管这一改进并不十分显著。

在图 22 中，我们详细展示了三个超参数的影响：补丁大小、调度和斜坡周期。不出所料，增加分配给较大补丁大小的时间比例（例如通过设置 schedule = 90%），会以提高计算能力为代价降低准确率。

I. 跨尺度下 token 表示间余弦相似度的进一步分析

在第 6节中，我们测量了种子 token 在某一尺度下的表示与其他尺度下其他 token 表示之间的余弦相似度，结果表明在其他尺度下最相似的 token 是那些表示相同空间位置的 token。在图 24中，我们提供了其他网格大小和来自额外块的种子 token 的补充结果。这些结果与正文中的结果一致。

J. 跨尺度集成 FlexiViT 预测

我们通过对 FlexiViT 模型的 Logit 进行平均来实现集成。[7] 在集成所有模型时，我们在 ImageNet-21K 验证集上达到了 51.7% 的准确率 @1，这略低于在最大网格大小/最小补丁大小下实现的准确率（52.0%）。

我们进一步在图 26中探索了成对模型的集成。在大尺寸补丁下评估的模型之间的一致性相对较低，在最大两个补丁尺寸（/48 和/40）下评估的模型仅在 67.4% 的样本上达成一致（图 26中间）。尽管如此，集成这些模型并未比单独使用 FlexiViT-B/40 带来准确率提升；两种策略均实现了 45.8% 的 ImageNet-21K 准确率 @1（图 26左侧）。

在比较不同尺度下 FlexiViT 预测集成的计算成本与单一尺度应用 FlexiViT 时，单一尺度几乎总是更优（图 26 右）。唯一一个两尺度集成在相同计算成本下准确率超过单一尺度的配置是 FlexiViT-B/10 和/12 的集成，它们的总计算成本与 FlexiViT-B/8 相近（/10 + /12: 186.8 GFLOPs；/8: 184.5 GFLOPs）。该集成在准确率上略有提升（52.1% 对 52.0%），但这一改进在统计上可能不显著，且在实际应用中意义不大。

K. FlexiViT 的形状与纹理偏好

当面对形状与纹理相冲突的图像时，经过 ImageNet 训练的模型倾向于生成与其纹理相匹配的标签，而人类则更倾向于分配与其形状相符的标签 [18]。我们使用与 [18] 相同的数据集评估了 FlexiViT，该数据集是通过风格迁移生成的。在 [18] 构建的数据集中，图像同时具有形状和纹理标签。我们将形状准确率定义为 top-1 预测与形状标签匹配的图像百分比，纹理准

[7]我们还探索了基于输出概率平均的集成方法，发现其结果几乎相同，但平均而言略逊一筹。

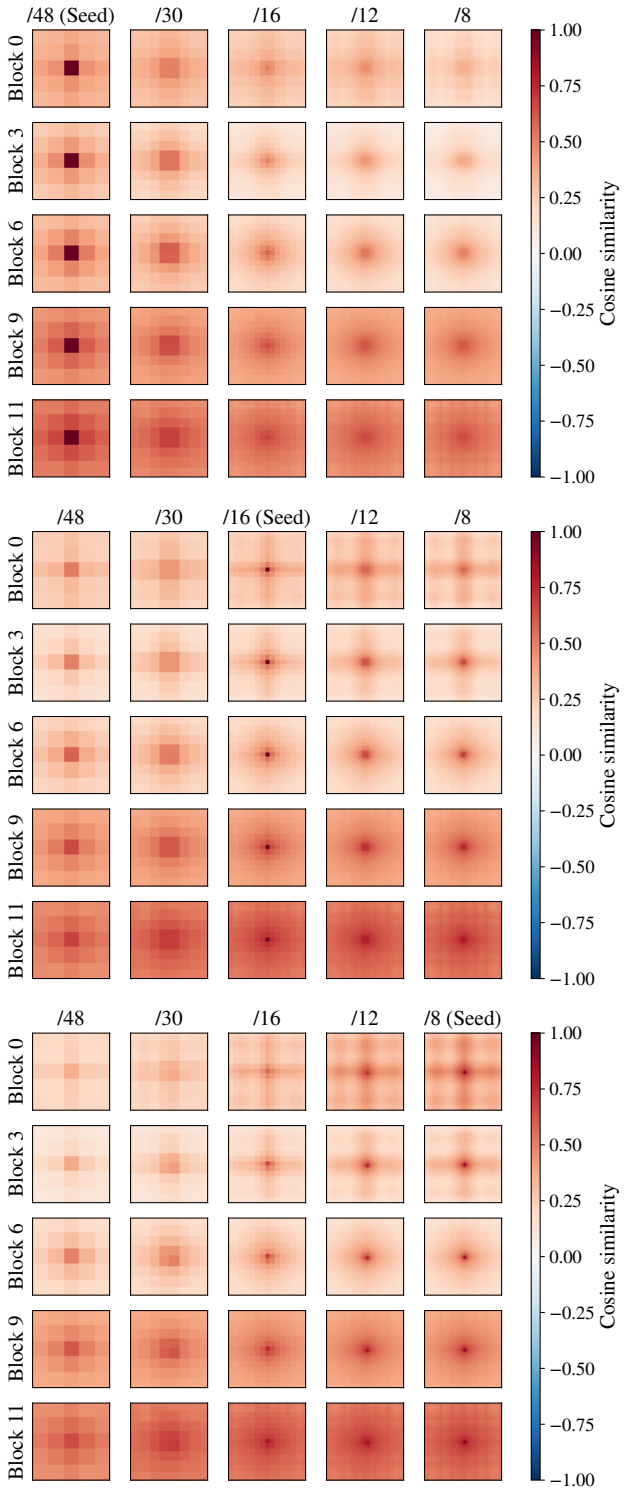

图 24. **跨尺度 token 表示相似性的补充分析**。每一行展示了一个块中，某一尺度下特征图中心的种子 token 与不同尺度下 token 之间的余弦相似度。在上方的图表组中，种子取自 FlexiViT-B/48；中间组取自 FlexiViT-B/16；底部组则取自 FlexiViT-B/8。

确率定义为 top-1 预测与纹理标签匹配的图像百分比。如 [18] 所述，我们通过将根据形状标签分类的图像数量与根据形状或纹理标签正确分类的图像数量之比转换为百分比来定义形状偏差。换句话说，形状偏差是形状准确率与形状和纹理准确率之和的比率 ×100%。为了在 [18] 的数据集上评估 ImageNet-21K 模型，我们使用 [18] 提供的从 WordNet ID 到数据集类别的映射。我们选择存在映射的 ImageNet-21K 类别中的 top-1 类别。

在图 25中，我们展示了较大的 patch 大小相较于较小的 patch 大小会导致更大的形状偏差。然而，较大的 patch 大小之所以具有更大的形状偏差，主要是因为它们的纹理准确率较低，而不是因为它们的形状准确率较高。FlexiViT-B/16 和 FlexiViT-B/30 的形状偏差与在单一尺度下训练的 ViT-B/16 和 ViT-B/30 模型的形状偏差相似（星号表示）。

L. 注意力相关性

我们提供了与主论文中图 13 相同的图像的注意力相关性图 [9]，但针对图像中存在的所有三个类别，如图 28 所示。

我们进一步提供了从unsplash.com中随机选择的 10 张免版税图像的注意力相关性图，这些图像对应于

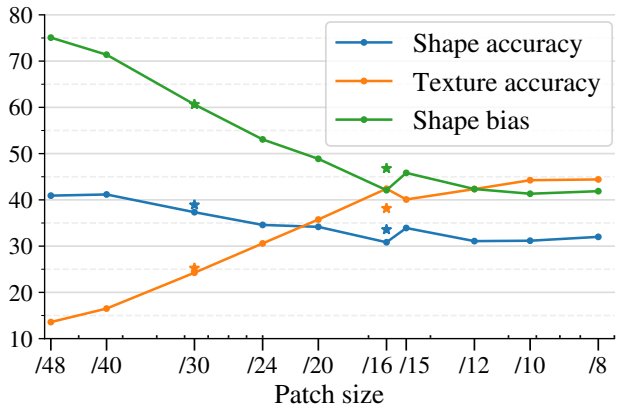

图 25. FlexiViT **的形状与纹理偏好**。线条表示 FlexiViT 在不同尺度下评估的值；星号表示在单一尺度下训练的基准 ViT 模型的值。形状与纹理准确率是模型针对形状和纹理标签预测的 top-1 准确率。形状偏好是指分类器在正确分类图像（无论是通过形状还是纹理）时，依据形状进行分类的图像所占百分比。

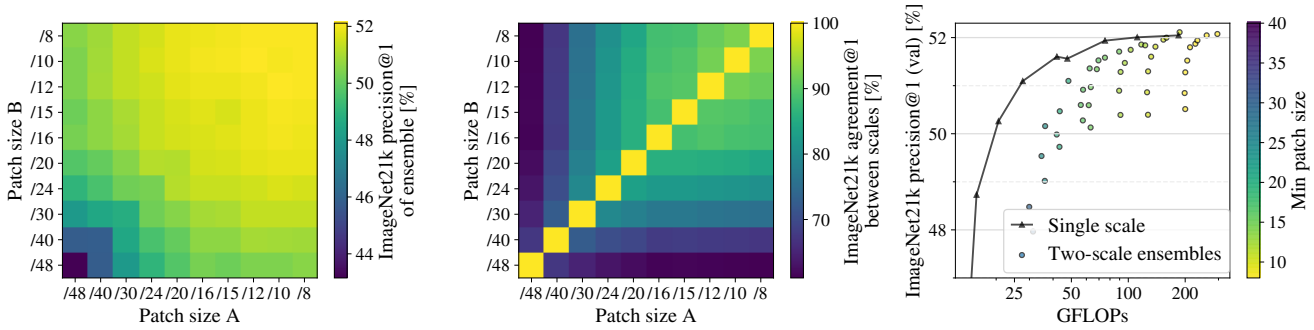

图 26. FlexiViT-B **在不同尺度组合下的集成准确率与一致性评估**。左图：不同尺度间集成的准确率。中图：各尺度间 top-1 预测类别的一致性。右图：单尺度配置与双尺度集成的准确率对比计算成本。尽管在较小 patch 大小时一致性相对较低，但集成带来的收益有限。

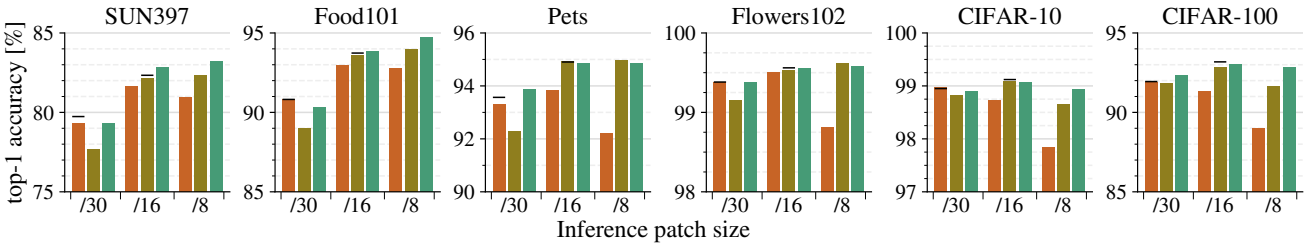

图 27. **更多迁移结果**。图例与图 9 相同。主要结论是，在广泛的图像分类任务中，结果在性质上保持一致。

所有模型在图 32末尾附录中预测的类别。

M. 灵活化迁移学习

在灵活化迁移学习时，我们运行与迁移预训练 FlexiViT 模型完全相同的设置，如第 E.1节所述，唯一的区别是现在在迁移过程中也随机化 patch 大小。

这一细微调整与使用预训练的 FlexiViT 模型（绿色柱状图）相结合时展现出良好的协同效应，甚至在一定程度上实现了在迁移过程中对普通 ViT 模型的灵活化处理（橙色与橄榄色柱状图）。

N. 开放词汇检测（OWL-ViT）的灵活化

为了灵活化 OWL-ViT（第 5.3节），我们采用 LiT-uu 骨干网络 [70]，即采用 CLIP 风格的模型，其中图像与文本编码器通过对比预训练共同优化（如 OWL-ViT 论文 [40] 所述）。我们按照算法 1的描述灵活化检测训练，并使用分辨率 720×720。在灵活检测训练中，我们采用从 48^2 到 12^2 的 patch 大小，即省略 10^2 和 8^2，以避免在高分辨率下占用过多内存。其他训练设置与 OWL-ViT 论文 [40] 保持一致。我们发现，同时灵活化图像-文本预训练与检测训练（图 11，浅绿色线）比仅灵活化检测训练效果略优。

在 ELEVATER [34] 数据集集合上进行评估时，我们使用了图像-文本预训练和检测训练均经过灵活化的模型（图 31）。

O. 关于灵活深度和步幅替代方案的详细信息

Common setup 设置大致遵循第 3.5 节中介绍的设置：从 [53] 蒸馏 ViT-B/8 模型，同时将其用于学生的初始化。蒸馏是在 ImageNet-21k 上进行的，标签被忽略，学生和教师之间的 KL 散度是唯一的损失，遵循 [5]。训练进行了 90 轮次，使用了块大小的均匀采样。

Flexible stride 我们训练了一个 FlexiViT 模型变体，在提取图像块时，我们灵活地改变窗口步幅，但将块大小固定为 32×32 。为了完美匹配默认网格大小，并完全覆盖整个图像并避免填充，我们进行最小必要的图像调整。例如，要获得网格大小 8×8 ，我们将图像调整为大小 242×242 （从 240×240 ），并应用步幅 30，或者要获得网格大小 24×24 ，我们将图像调整为大小 239×239 并应用步幅 9。

Flexible depth For every batch, we sample a depth

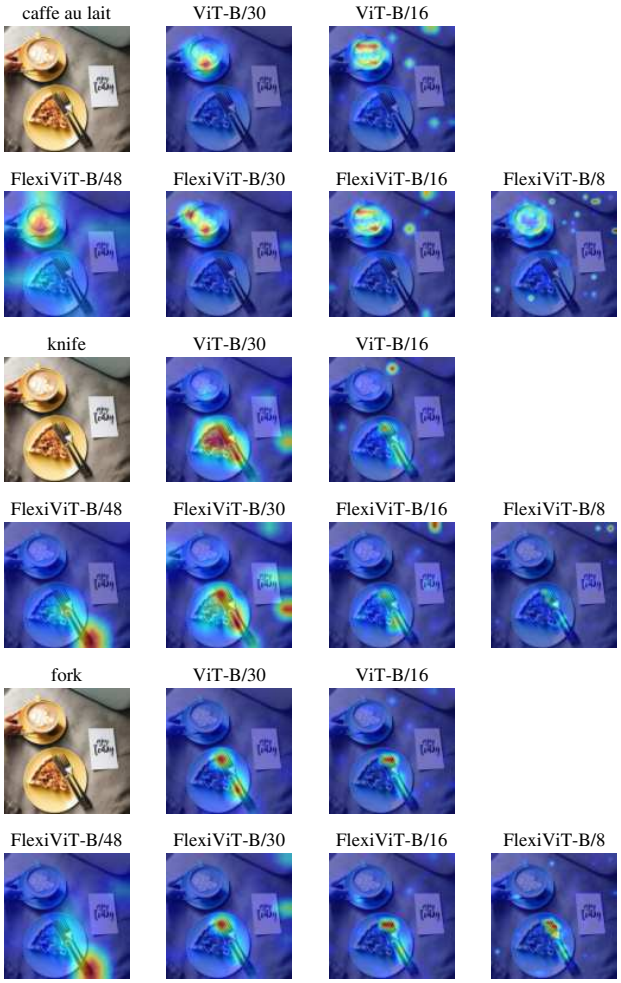

图 28. **注意力相关图**展示了与图 13 中相同的示例，针对图像中存在的三个不同物体（“牛奶咖啡”、“刀” 和 “叉”）。

d uniformly from $\{3, 5, 9, 12\}$ and perform a forward pass up to layer d. We then apply the classification head, which is shared across all depths, to the class token of layer d and compute the loss. We also explored sampling a depth per-example, but this led to unstable training except when sampling d from all layers $\{3, 4, \ldots, 12\}$. Finally, we also tried using a head per depth as well as a class token per depth, which did not lead to any significant improvement.

P. 图 2 中的 GFLOPs

继效率误称 [15] 之后，我们也提供了以 GFLOPs 为 x 轴而非推理时间的图 2副本，即图 29。这证实了

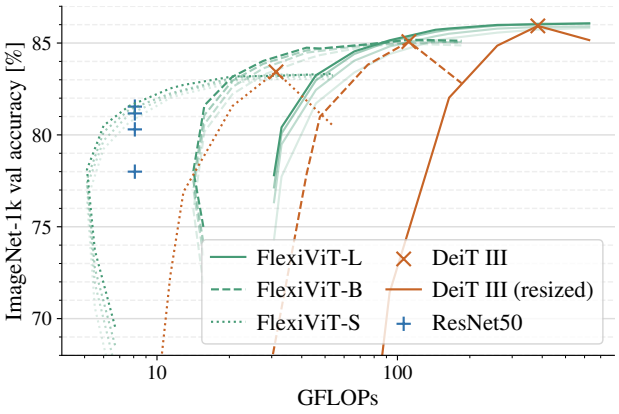

图 29. **图 2 使用** GFLOPs。

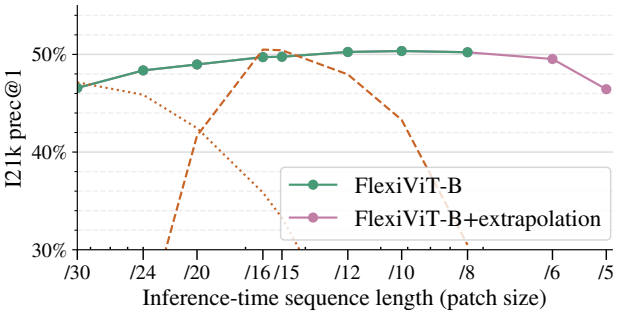

图 30. **补丁大小外推法**。我们评估了在绿色补丁大小上训练的模型在训练期间未见过的补丁大小范围内的表现，并用粉色绘制这些结果。性能逐渐下降意味着模型无法对训练期间见过的补丁大小或序列长度进行外推。

在某些情况下，FLOPs 并不总是直接对应于实际运行时间。

Q. 推测补丁大小

我们从主论文的第 3.3 节中提取模型，并在更小的 patch 大小下进行推理，参见图 30。我们观察到性能开始缓慢下降。这意味着模型并未学会泛化到训练期间未见过的 patch 大小或序列长度。请注意，我们并不声称具备外推能力，而是表明将多种 patch 大小训练到一个单一模型中不会导致质量损失。

R. 完整表格结果

我们提供了包含主论文中所有图表数值结果的 Tabs. 1 to 13。

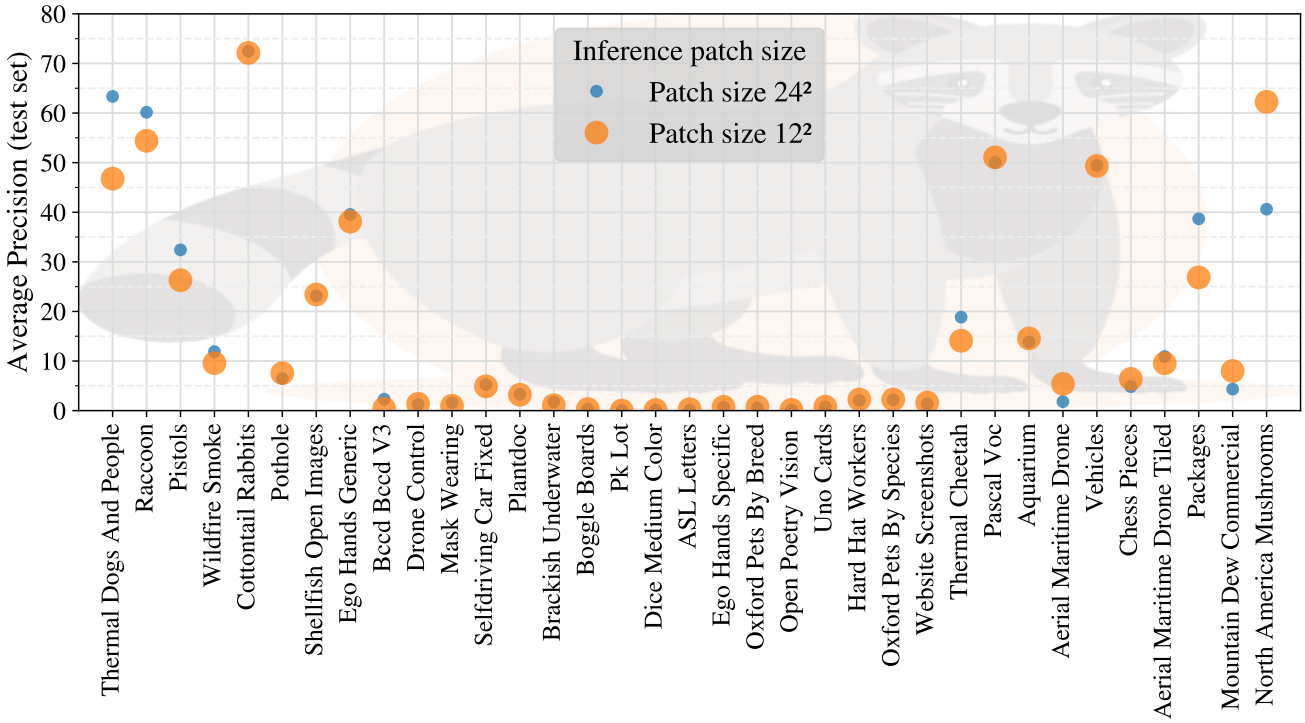

图 31. **推理时调整 patch 大小可以提升性能**。单个 OWL-FlexiViT-B 模型在 ELEVATER 基准的 35 个不同检测任务上以两种不同的 patch 大小（30^2 和 16^2）进行评估。 [34]。任务按照 patch 大小 30^2 和 16^2 在验证集上的性能差异排序；y 轴显示在测试集上的性能。不同任务的最佳 patch 大小各异，因此在验证集上调整 patch 大小将平均测试性能从 15.63 AP（patch 大小 16^2）或 16.19 AP（patch 大小 30^2）提升至 16.62 AP。

S. 配置文件用于图 2

Algorithm S 展示了 big_vision[8] 用于训练图 2 中 FlexiViT 模型的配置。

[8] https://github.com/google-research/big_vision

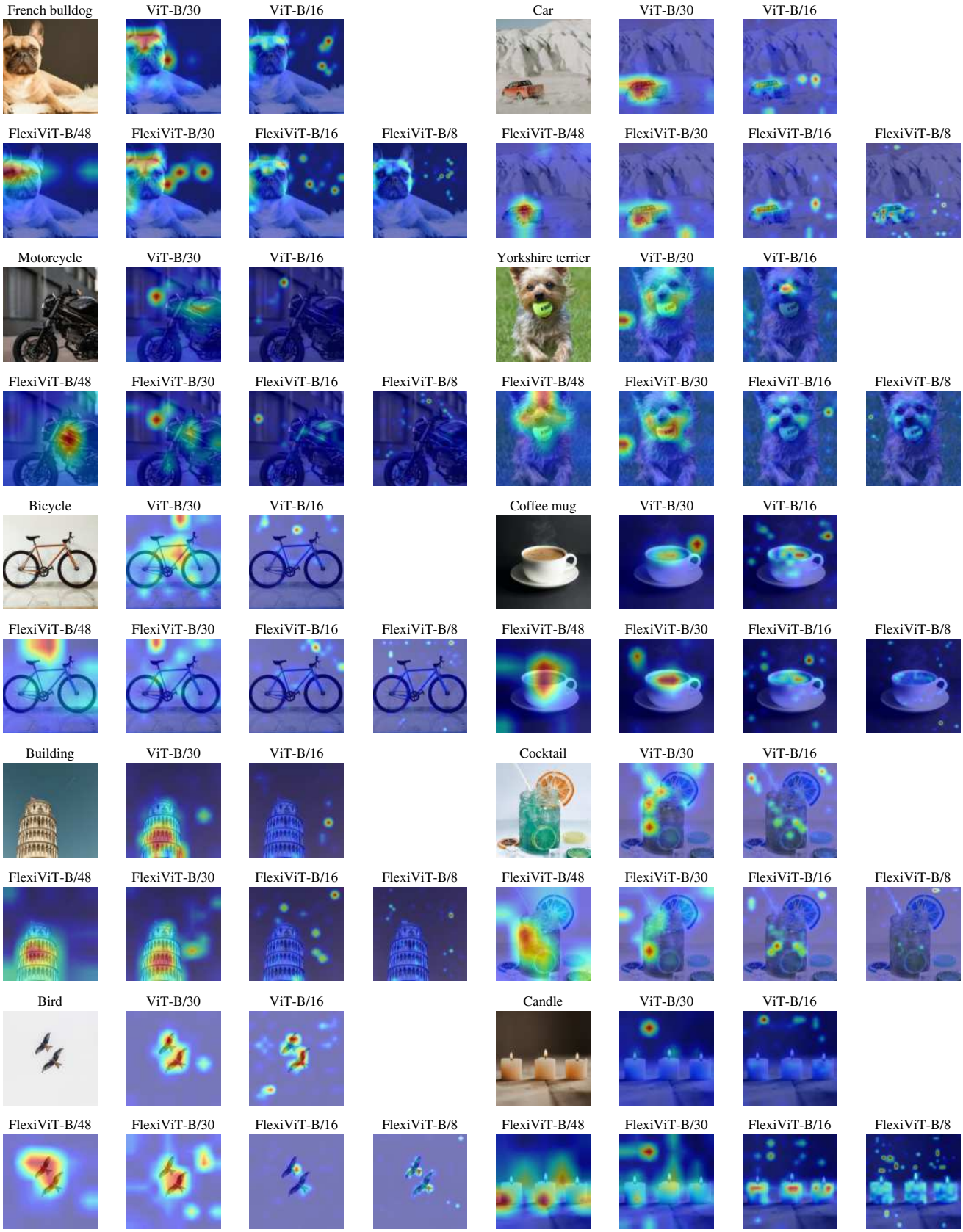

图 32. **注意力相关性**（如 [9] 中所示）在 ViT 和 FlexiViT 中，随着 patch 大小的不同，可能会发生显著变化。

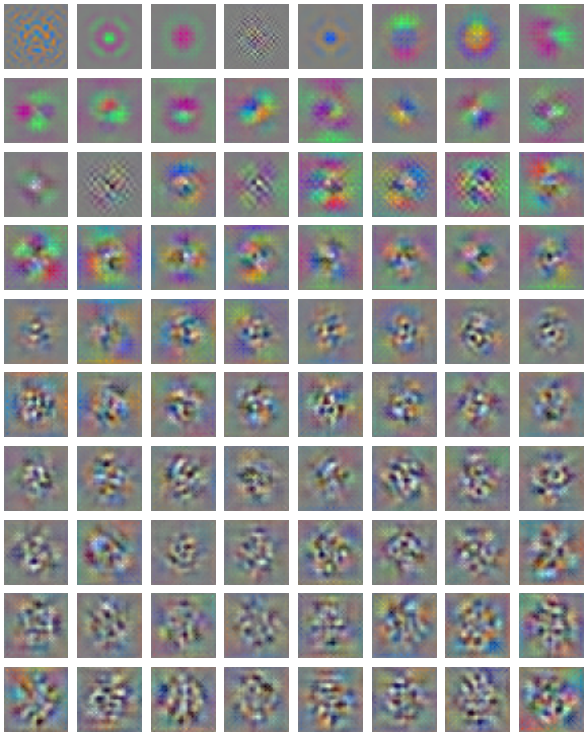

图 33. FlexiViT 中 32x32 补丁潜在嵌入权重的首 80 个 PAC 成分。

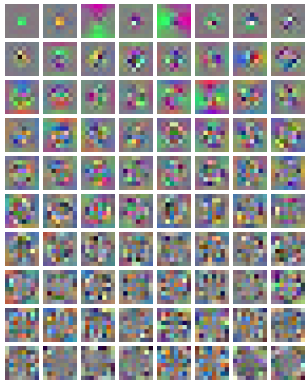

图 34. 图 33 中补丁嵌入权重的前 80 个 PCA 成分，双线性调整为 8x8 大小。

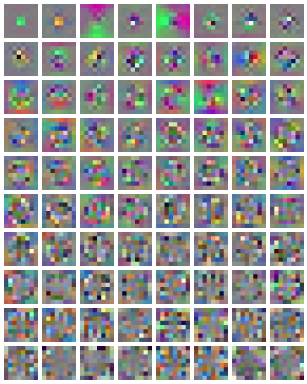

图 35. 图 33 中补丁嵌入权重的首 80 个 PCA 成分，调整为 8x8 大小。

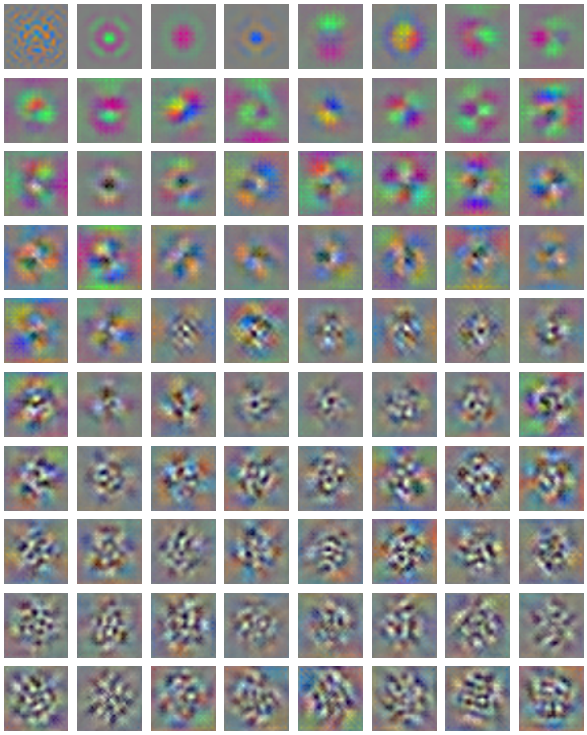

图 36. 图 33 中补丁嵌入权重的首 80 个 PCA 成分，经过双线性调整至 48x48 大小。

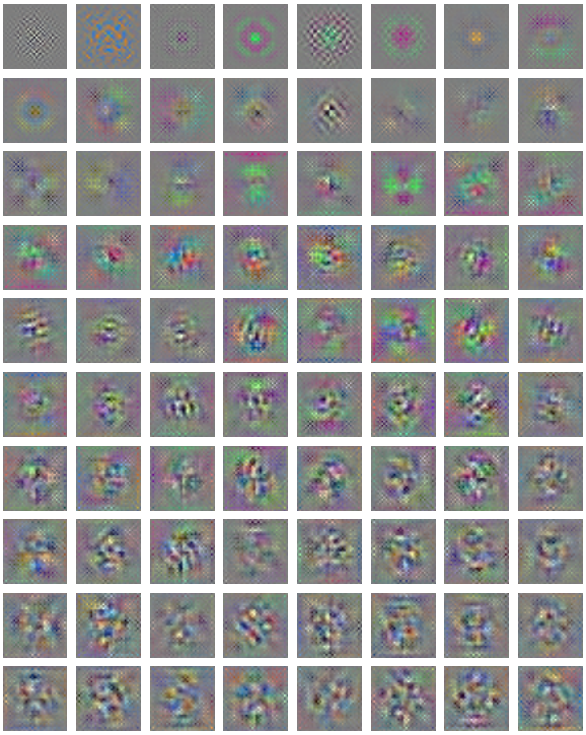

图 37. 图 33 中补丁嵌入权重的前 80 个 PCA 成分，已调整为 48x48 大小。

```python
def get_config(arg=None):
  """Config for training FlexiViT on ImageNet1k."""
  c = bvcc.parse_arg(arg, variant='B')
  c.total_epochs = 90
  c.num_classes = 1000
  c.loss = 'softmax_xent'

  c.input = {}
  c.input.data = dict(
      name='imagenet2012',
      split='train[:99
  )
  c.input.batch_size = 1024
  c.input.shuffle_buffer_size = 250_000

  c.log_training_steps = 50
  c.ckpt_steps = 1000

  # Model section
  c.student_name = 'proj.flexi.vit'
  c.student_init = f'deit_3_{c.variant}_384_1k'
  c.student = dict(variant=c.variant,
                   pool_type='tok',
                   patch_size=(16, 16))

  c.teachers = ['prof']
  c.prof_name = 'vit'
  c.prof_init = f'deit_3_{c.variant}_384_1k'
  c.prof = dict(variant=c.variant,
                pool_type='tok',
                patch_size=(16, 16))

  pp_label = (
      '|onehot(1000, key="{lbl}", key_result="labels")'
      '|keep("image", "prof", "labels")')
  c.input.pp = (
      'decode|inception_crop|flip_lr'
      '|copy("image", "prof")'
      '|resize(240)'
      '|vgg_value_range'
      '|resize(384, key="prof")'
      '|vgg_value_range(key="prof")'
      + pp_label.format(lbl='label'))
  pp_eval_both = (
      'decode|copy("image", "prof")|'
      f'|resize({240//7*8})'
      '|central_crop(240)'
      '|vgg_value_range'
      f'|resize({384//7*8}, key="prof")'
      '|central_crop(384, key="prof")'
      '|vgg_value_range(key="prof")|')
  pp_eval_student = (
      'decode'
      f'|resize({240//7*8})|central_crop({240})'
      '|value_range(-1, 1)')
  pp_eval_prof = (
      'decode'
      f'|resize({384//7*8})|central_crop(384)'
      '|vgg_value_range(outkey="prof")')

  # Distillation settings
  c.mixup = dict(p=1.0, n=2)
  c.distance = 'kl'
  c.distance_kw = dict(t=1.0)

  # Optimizer section
  c.grad_clip_norm = 1.0
  c.optax_name = 'scale_by_adam'
  c.optax = dict(mu_dtype='bfloat16')

  c.lr = 1e-4
  c.wd = 1e-5
  c.schedule = dict(
      warmup_steps=5000,
      decay_type='cosine')

  # Define the flexible model params:
  c.flexi = dict()
  c.flexi.seqhw = dict(
      # The settings to sample from.
      # Corresponding patch-sizes at 240px:
      # 48, 40, 30, 24, 20, 16, 15, 12, 10, 8
      v=(5, 6, 8, 10, 12, 15, 16, 20, 24, 30),
      # The probs/weights of them (uniform):
      p=(1, 1, 1, 1, 1, 1, 1, 1, 1, 1),
  )

  ####
  # All the rest is just evaluations.
  minitrain = 'train[:2
  minival = 'train[99

  def get_eval(s, split, dataset='imagenet2012'):
    return dict(
        type='classification',
        pred=f'student_seqhw={s}',
        data=dict(name=dataset, split=split),
        pp_fn=(pp_eval_student +
               pp_label.format(lbl='label')),
        loss_name='sigmoid_xent',
        log_percent=0.05,
        cache_final=False,
```

```
    )

  c.evals = {}
  for s in c.flexi.seqhw.v:
    c.evals[f'student_minitrain_{s:02d}'] = \
        get_eval(s, minitrain)
    c.evals[f'student_minival_{s:02d}'] = \
        get_eval(s, minival)
    c.evals[f'student_val_{s:02d}'] = \
        get_eval(s, 'validation')
    c.evals[f'student_v2_{s:02d}'] = \
        get_eval(s, 'test', 'imagenet_v2')
    c.evals[f'student_a_{s:02d}'] = \
        get_eval(s, 'test', 'imagenet_a')
    c.evals[f'student_r_{s:02d}'] = \
        get_eval(s, 'test', 'imagenet_r')
    c.evals[f'student_real_{s:02d}'] = \
        get_eval(s, 'validation',
                 'imagenet2012_real')
    c.evals[f'student_real_{s:02d}'].pp_fn = (
        pp_eval_student +
        pp_label.format(lbl='real_label'))

  # A bunch more evals here ...

  return c
```

表 1. 1200 轮 ImageNet-1k 专用训练的得分，来自图 2。

Model	Eps	PS	Val	ReaL	v2	-A	-R
FlexiViT-S	1200	48^2	69.6	76.1	55.5	3.3	24.1
FlexiViT-S	1200	40^2	73.7	80.2	60.3	4.7	26.4
FlexiViT-S	1200	30^2	78.1	84.3	65.1	7.4	29.2
FlexiViT-S	1200	24^2	80.5	86.3	68.4	9.4	30.5
FlexiViT-S	1200	20^2	81.6	87.2	70.3	12.2	31.8
FlexiViT-S	1200	16^2	82.5	87.9	71.7	15.0	31.4
FlexiViT-S	1200	15^2	82.7	88.1	71.8	15.7	32.9
FlexiViT-S	1200	12^2	83.2	88.4	72.7	17.8	32.9
FlexiViT-S	1200	10^2	83.2	88.4	72.9	19.4	33.0
FlexiViT-S	1200	8^2	83.3	88.5	72.9	19.3	32.6
FlexiViT-B	1200	48^2	75.0	80.5	61.1	5.7	28.1
FlexiViT-B	1200	40^2	78.0	83.3	64.7	7.5	30.0
FlexiViT-B	1200	30^2	81.6	86.5	69.7	11.4	33.0
FlexiViT-B	1200	24^2	83.2	87.7	71.7	15.5	34.5
FlexiViT-B	1200	20^2	84.0	88.4	73.0	18.4	35.6
FlexiViT-B	1200	16^2	84.7	88.8	74.0	21.7	35.8
FlexiViT-B	1200	15^2	84.7	88.8	74.3	22.7	36.7
FlexiViT-B	1200	12^2	84.9	89.1	74.8	25.3	37.1
FlexiViT-B	1200	10^2	85.2	89.2	75.0	26.7	37.2
FlexiViT-B	1200	8^2	85.1	89.2	74.9	27.1	37.2
FlexiViT-L	1200	48^2	77.8	83.3	63.9	7.1	30.0
FlexiViT-L	1200	40^2	80.4	85.6	67.2	9.9	32.7
FlexiViT-L	1200	30^2	83.2	87.9	70.8	14.8	35.9
FlexiViT-L	1200	24^2	84.5	88.8	73.6	19.9	38.1
FlexiViT-L	1200	20^2	85.1	89.4	74.9	23.5	39.4
FlexiViT-L	1200	16^2	85.7	89.7	76.0	28.6	39.6
FlexiViT-L	1200	15^2	85.8	89.9	76.0	29.1	40.6
FlexiViT-L	1200	12^2	86.0	90.0	76.5	32.0	40.8
FlexiViT-L	1200	10^2	86.0	90.0	76.8	33.6	40.9
FlexiViT-L	1200	8^2	86.1	90.0	76.7	34.1	41.2

表 2. 600 个 epoch 的 ImageNet-1k-only 运行的得分来自图 2。

Model	Eps	PS	Val	ReaL	v2	-A	-R
FlexiViT-S	600	48^2	68.6	75.1	54.2	3.2	23.9
FlexiViT-S	600	40^2	72.7	79.4	59.3	4.4	26.3
FlexiViT-S	600	30^2	77.6	83.8	64.6	6.9	29.0
FlexiViT-S	600	24^2	80.2	86.0	67.8	9.2	30.4
FlexiViT-S	600	20^2	81.4	87.0	69.9	11.5	31.5
FlexiViT-S	600	16^2	82.3	87.7	71.2	14.2	31.2
FlexiViT-S	600	15^2	82.5	87.9	71.5	15.1	32.7
FlexiViT-S	600	12^2	83.1	88.3	72.5	17.5	32.7
FlexiViT-S	600	10^2	83.3	88.5	72.7	19.5	32.7
FlexiViT-S	600	8^2	83.3	88.5	72.8	19.4	32.4
FlexiViT-B	600	48^2	74.1	80.0	60.2	5.3	27.7
FlexiViT-B	600	40^2	77.5	83.0	64.4	7.5	30.0
FlexiViT-B	600	30^2	81.1	86.1	68.9	11.2	32.7
FlexiViT-B	600	24^2	82.9	87.5	71.6	15.0	34.2
FlexiViT-B	600	20^2	83.9	88.2	72.6	17.5	35.4
FlexiViT-B	600	16^2	84.6	88.7	73.9	22.1	35.7
FlexiViT-B	600	15^2	84.7	88.8	73.9	22.6	36.6
FlexiViT-B	600	12^2	84.9	89.0	74.7	25.4	36.9
FlexiViT-B	600	10^2	85.1	89.2	74.8	26.9	37.2
FlexiViT-B	600	8^2	85.0	89.2	74.8	27.0	36.9
FlexiViT-L	600	48^2	77.1	82.6	62.7	7.2	30.1
FlexiViT-L	600	40^2	80.1	85.2	66.6	9.4	32.6
FlexiViT-L	600	30^2	83.0	87.7	71.0	14.6	36.0
FlexiViT-L	600	24^2	84.4	88.8	73.4	19.3	38.1
FlexiViT-L	600	20^2	85.1	89.4	74.7	22.5	39.3
FlexiViT-L	600	16^2	85.6	89.7	76.0	27.7	39.7
FlexiViT-L	600	15^2	85.7	89.8	75.9	28.3	40.8
FlexiViT-L	600	12^2	85.9	89.9	76.4	31.0	41.0
FlexiViT-L	600	10^2	86.1	90.0	76.6	32.8	41.1
FlexiViT-L	600	8^2	86.1	90.0	76.6	33.2	41.3

表 3. 300 轮 ImageNet-1k 专属训练的得分来自图 2。

Model	Eps	PS	Val	ReaL	v2	-A	-R
FlexiViT-S	300	48^2	67.4	74.0	53.2	2.8	23.5
FlexiViT-S	300	40^2	71.9	78.6	58.2	3.9	26.0
FlexiViT-S	300	30^2	77.2	83.5	64.2	6.5	29.0
FlexiViT-S	300	24^2	79.7	85.6	67.6	8.8	30.2
FlexiViT-S	300	20^2	81.1	86.7	69.6	11.1	31.4
FlexiViT-S	300	16^2	82.1	87.6	71.1	13.9	31.1
FlexiViT-S	300	15^2	82.4	87.9	71.3	14.9	32.7
FlexiViT-S	300	12^2	83.0	88.2	72.3	17.6	32.5
FlexiViT-S	300	10^2	83.2	88.3	72.9	19.3	32.4
FlexiViT-S	300	8^2	83.2	88.3	73.0	19.4	32.2
FlexiViT-B	300	48^2	73.4	79.3	59.7	4.9	27.4
FlexiViT-B	300	40^2	77.0	82.6	63.7	7.0	29.6
FlexiViT-B	300	30^2	80.6	85.8	68.4	10.4	32.7
FlexiViT-B	300	24^2	82.6	87.3	71.2	14.6	34.1
FlexiViT-B	300	20^2	83.6	88.1	72.5	17.3	35.1
FlexiViT-B	300	16^2	84.5	88.6	73.8	21.8	35.4
FlexiViT-B	300	15^2	84.6	88.7	73.9	22.5	36.5
FlexiViT-B	300	12^2	84.9	89.0	74.6	25.5	36.9
FlexiViT-B	300	10^2	85.0	89.1	74.8	27.3	37.0
FlexiViT-B	300	8^2	85.1	89.1	75.0	27.1	36.9
FlexiViT-L	300	48^2	76.3	81.8	62.4	6.5	30.2
FlexiViT-L	300	40^2	79.5	84.7	66.4	9.0	32.6
FlexiViT-L	300	30^2	82.4	87.3	70.5	13.7	35.9
FlexiViT-L	300	24^2	84.0	88.5	72.8	18.0	37.7
FlexiViT-L	300	20^2	84.8	89.1	74.3	21.7	39.2
FlexiViT-L	300	16^2	85.4	89.6	75.7	26.7	39.4
FlexiViT-L	300	15^2	85.5	89.7	75.8	28.1	40.6
FlexiViT-L	300	12^2	85.8	89.9	76.4	30.9	40.8
FlexiViT-L	300	10^2	85.9	89.9	76.8	32.7	41.0
FlexiViT-L	300	8^2	85.9	90.0	76.7	33.4	41.2

表 4. 90 个周期 ImageNet-1k 专用运行的得分来自图 2。

Model	Eps	PS	Val	ReaL	v2	-A	-R
FlexiViT-S	90	48^2	65.9	72.5	51.9	2.9	23.2
FlexiViT-S	90	40^2	70.6	77.3	56.9	3.5	26.0
FlexiViT-S	90	30^2	76.4	82.9	62.9	5.9	29.3
FlexiViT-S	90	24^2	79.2	85.3	66.8	7.8	30.6
FlexiViT-S	90	20^2	80.7	86.5	69.0	10.6	31.5
FlexiViT-S	90	16^2	82.0	87.5	70.9	13.9	31.7
FlexiViT-S	90	15^2	82.2	87.7	71.1	14.4	32.7
FlexiViT-S	90	12^2	82.8	88.1	72.0	17.3	32.5
FlexiViT-S	90	10^2	83.0	88.2	72.7	19.0	32.4
FlexiViT-S	90	8^2	83.0	88.3	72.7	19.4	32.2
FlexiViT-B	90	48^2	71.9	77.8	58.0	4.6	26.9
FlexiViT-B	90	40^2	75.9	81.6	62.2	6.1	29.3
FlexiViT-B	90	30^2	80.2	85.3	67.3	9.5	32.3
FlexiViT-B	90	24^2	82.2	87.0	70.2	13.1	34.1
FlexiViT-B	90	20^2	83.3	87.8	71.9	16.5	35.1
FlexiViT-B	90	16^2	84.1	88.4	73.2	21.3	35.2
FlexiViT-B	90	15^2	84.3	88.5	73.9	21.8	36.4
FlexiViT-B	90	12^2	84.8	88.8	74.4	25.2	36.8
FlexiViT-B	90	10^2	85.0	89.0	74.5	27.3	36.9
FlexiViT-B	90	8^2	84.9	89.0	74.7	27.7	36.6
FlexiViT-L	90	48^2	74.3	80.0	60.2	5.6	29.4
FlexiViT-L	90	40^2	77.7	83.2	64.5	7.7	32.2
FlexiViT-L	90	30^2	81.7	86.7	69.4	12.1	35.3
FlexiViT-L	90	24^2	83.4	88.0	72.4	17.1	37.3
FlexiViT-L	90	20^2	84.4	88.8	73.9	20.9	38.9
FlexiViT-L	90	16^2	85.1	89.3	75.4	26.5	39.4
FlexiViT-L	90	15^2	85.3	89.5	75.6	27.2	40.3
FlexiViT-L	90	12^2	85.6	89.7	76.3	31.2	40.5
FlexiViT-L	90	10^2	85.7	89.8	76.7	33.1	40.7
FlexiViT-L	90	8^2	85.8	89.9	76.6	33.7	40.6

表 5. 图 3 的数值数据。

Model	/48	/40	/30	/24	/20	/16	/15	/12	/10	/8
Flexi	39.5	43.2	46.6	48.4	49.0	49.7	49.8	50.2	50.3	50.2
B/16	0.0	0.1	2.4	21.6	41.7	50.5	50.4	47.9	43.3	30.5
B/30	14.0	30.2	47.1	45.9	42.5	35.9	33.3	21.0	11.9	2.9

表 6. 图 5 的数值数据。

		/5	/6	/8	/10	/12	/15	/16	/20	/24	/30
Top-1 accuracy											
T-init	90	40.8	43.8	47.9	49.4	50.5	51.3	51.4	51.9	51.9	52.0
	300	43.1	45.9	48.7	50.3	51.1	51.6	51.6	51.9	52.0	52.0
	1000	44.1	46.6	49.2	50.6	51.2	51.9	51.8	52.1	52.3	52.2
R-init		41.5	44.2	47.0	48.5	48.9	49.6	49.8	50.0	50.1	50.0
None		40.6	43.4	46.6	48.1	48.9	49.7	49.7	50.0	50.2	50.1
Teacher		52.2									
Top-1 agreement											
T-init	90	56.0	61.9	69.5	74.7	78.1	81.1	81.8	83.8	84.5	84.4
	300	59.6	65.7	73.0	77.3	80.1	82.8	82.9	84.2	84.5	84.5
	1000	62.0	67.5	74.4	78.6	81.3	83.4	83.7	84.6	85.2	85.0
R-init		56.4	60.7	66.2	69.0	70.7	72.1	72.4	72.8	73.1	73.1
CKA similarity											
T-init	90	.65	.69	.76	.80	.84	.86	.87	.88	.89	.89
	300	.68	.72	.78	.82	.85	.86	.87	.87	.88	.87
	1000	.68	.72	.78	.81	.83	.85	.85	.86	.86	.86
R-init		.41	.43	.45	.47	.48	.49	.50	.50	.50	.50

表 7. 图 7 的数值数据。

Method	/30→	/30F→	/16→	/16F→
Clf SUN397	79.1	79.7	82.3	82.5
Clf Food101	90.4	90.8	93.7	94.1
Clf Pets	93.6	93.6	94.9	94.9
Clf Flowers102	99.4	99.4	99.6	99.6
Clf CIFAR-10	98.8	99.0	99.1	99.1
Clf CIFAR-100	92.3	91.9	93.2	93.3
UViM coco PQ	24.8	24.1	30.5	34.3
OWL-ViT lvis AP	18.8	18.4	22.7	23.4
LiT i2t coco	42.2	41.6	44.4	45.8
Seg (lin) City mIoU	61.0	61.1	69.3	70.0
Seg (lin) ADE mIoU	43.1	43.5	46.1	47.5

表 8. 图 8 的数值数据。

/40	/30	/24	/20	/15	/12	/10	/8
ViT-B/30 transferred at /30							
78.3	82.4	83.0	82.7	80.4	75.1	66.7	58.6
FlexiViT-B transferred at /30							
78.0	81.8	83.4	84.3	85.0	85.2	85.3	84.9
FlexiViT-B transferred at /16							
77.0	81.2	83.2	84.5	85.5	85.9	86.0	85.7
FlexiViT-B transferred at /8							
76.2	80.6	82.8	84.1	85.3	85.8	86.1	86.4

表 9. 图 9 的数值数据。

Model	Transfer	SUN	Food	Pet	Flow	C10	C100
Evaluated at /30							
ViT-B/30	/30	79.7	90.8	93.6	99.4	99.0	91.9
ViT-B/30	Flexi	79.3	90.8	93.3	99.4	99.0	91.9
ViT-B/16	Flexi	77.7	89.0	92.3	99.2	98.8	91.8
FlexiViT-B	Flexi	79.3	90.3	93.9	99.4	98.9	92.3
Evaluated at /16							
ViT-B/16	/16	82.3	93.7	94.9	99.6	99.1	93.2
ViT-B/30	Flexi	81.6	93.0	93.8	99.5	98.7	91.3
ViT-B/16	Flexi	82.1	93.6	94.9	99.5	99.1	92.8
FlexiViT-B	Flexi	82.8	93.8	94.8	99.6	99.1	93.0
Evaluated at /8							
ViT-B/30	Flexi	80.9	92.7	92.2	98.8	97.8	89.0
ViT-B/16	Flexi	82.4	94.0	95.0	99.6	98.7	91.6
FlexiViT-B	Flexi	83.2	94.7	94.9	99.6	98.9	92.8

表 10. 图 10 的数值数据。

Base	LiT	/48	/30	/24	/16	/12	/10	/8
ViT-B/30	/30	46.7	65.2	65.2	64.5	30.9	10.3	3.2
ViT-B/16	/16	3.2	42.7	60.2	71.9	61.6	45.7	57.8
FlexiViT-B	/30	49.0	59.8	67.0	72.5	72.6	74.7	74.1
FlexiViT-B	/16	48.2	62.6	66.6	73.3	73.1	74.5	75.0
FlexiViT-B	Flexi	51.0	62.5	69.2	73.4	74.5	75.5	75.1

表 11. 图 11 的数值数据。

Base	OWL	/48	/40	/30	/24	/20	/16	/12
LiT-B/30	/30	6.2	10.8	20.6	15.6	7.8	1.7	0.3
LiT-B/30	/16	0.1	0.5	4.1	9.5	16.7	26.8	15.0
LiT-B/30	Flexi	15.7	17.8	21.2	23.0	24.2	25.6	26.0
FlexiLiT-B	Flexi	16.0	18.5	21.5	23.5	24.9	26.7	27.1

表 12. 图 4 的数值数据。

Resize	/2	/4	/6	/8	/10	/12	/14	/16	/18	/20	/22	/24	/26	/28	/30	/32
PI	10.3	44.5	48.9	52.4	52.4	52.4	52.4	52.3	52.3	52.4	52.3	52.4	52.3	52.3	52.4	52.4
Area	24.5	42.6	46.1	52.4	47.7	48.3	48.6	48.1	48.7	48.4	48.5	48.7	48.5	48.5	48.6	48.4
Norm	10.8	40.7	46.4	52.4	45.6	44.1	42.0	39.4	37.1	33.4	29.8	26.1	22.0	18.1	14.8	11.9
Vanilla	25.5	41.6	45.6	52.4	42.5	37.2	25.5	12.8	5.4	1.9	0.6	0.2	0.1	0.0	0.0	0.0

表 13. 图 14 的数值数据。

Setting	Params	GFLOPs	Speed	Prec
Patch FlexiViT-B				
Eval at patch /30	87.5M	15.7	2745	47.9
Eval at patch /24	87.5M	20.6	2022	49.4
Eval at patch /20	87.5M	27.7	1135	50.5
Eval at patch /16	87.5M	41.9	806	51.3
Eval at patch /15	87.5M	47.6	595	51.4
Eval at patch /12	87.5M	75.3	362	51.9
Eval at patch /10	87.5M	111.5	246	51.9
Eval at patch /8	87.5M	184.5	128	52.0
Stride FlexiViT-B/32				
Eval at stride 30	87.8M	11.7	2317	47.5
Eval at stride 23	87.8M	18.2	1768	49.3
Eval at stride 19	87.8M	26.3	1041	50.3
Eval at stride 15	87.8M	41.7	750	51.0
Eval at stride 14	87.8M	47.6	563	51.2
Eval at stride 11	87.8M	76.4	347	51.5
Eval at stride 9	87.8M	113.7	235	51.6
Eval at stride 7	87.8M	188.4	124	51.7
Depth FlexiViT-B/8				
Eval at depth 3	22.1M	46.3	403	35.2
Eval at depth 6	43.4M	92.2	216	44.8
Eval at depth 9	64.6M	138.2	147	48.7
Eval at depth 12	85.9M	184.2	112	51.0